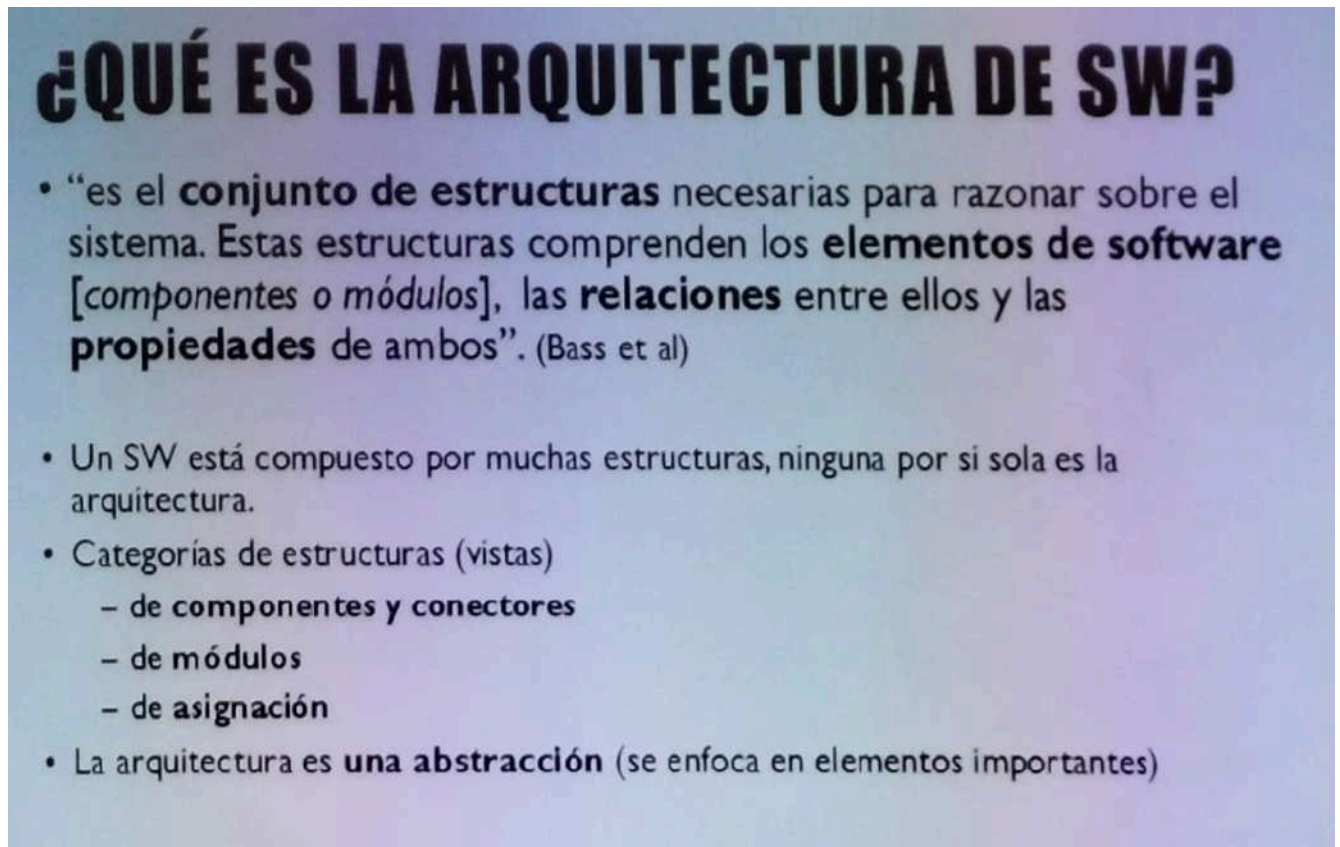


CLASE - 03/09

ARQUITECTURA

¿QUÉ ES LA ARQUITECTURA DE SW?



¿QUÉ ES LA ARQUITECTURA DE SW?

- “es el **conjunto de estructuras** necesarias para razonar sobre el sistema. Estas estructuras comprenden los **elementos de software** [*componentes o módulos*], las **relaciones** entre ellos y las **propiedades** de ambos”. (Bass et al)
- Un SW está compuesto por muchas estructuras, ninguna por sí sola es la arquitectura.
- Categorías de estructuras (vistas)
 - de **componentes y conectores**
 - de **módulos**
 - de **asignación**
- La arquitectura es **una abstracción** (se enfoca en elementos importantes)

-(hay que tomar como que está hablando de arquitectura de sw)

-elementos de sw es una forma abstracta de dominar a los **componentes o módulos**

-tienen 3 estructuras principales, en las 2 primeras están presentes y en la tercera directamente ya se unen

-una arquitectura es el modelo de sw pero con el nivel necesario de abstracción, no llega a llegar a cierto nivel de detalle (eso sería hacer diseño en detalle). Llega hasta el nivel de comprender el sistema en su totalidad

-imaginen como que a un sw lo puedo mirar desde diferentes perspectivas

-son aquellos elementos que son más importantes para que nos hagan razonar sobre el sistema

Del libro:

No todas las estructuras son arquitecturales.

Una estructura es arquitectural si soporta razonamiento acerca del sistema y sus propiedades.

El razonamiento debe centrarse en un atributo del sistema que sea importante para alguno o algunos interesados.

Algunos ejemplos son funcionalidad del sistema frente a fallos o intentos de tirarlo, la facilidad de realizar cambios específicos en el sistema, responsividad a peticiones de usuarios, y otros requisitos NO funcionales.

La arquitectura está más enfocada en la parte pública de un sistema como interfaces más que en atributos privados que se refieren a implementaciones internas.

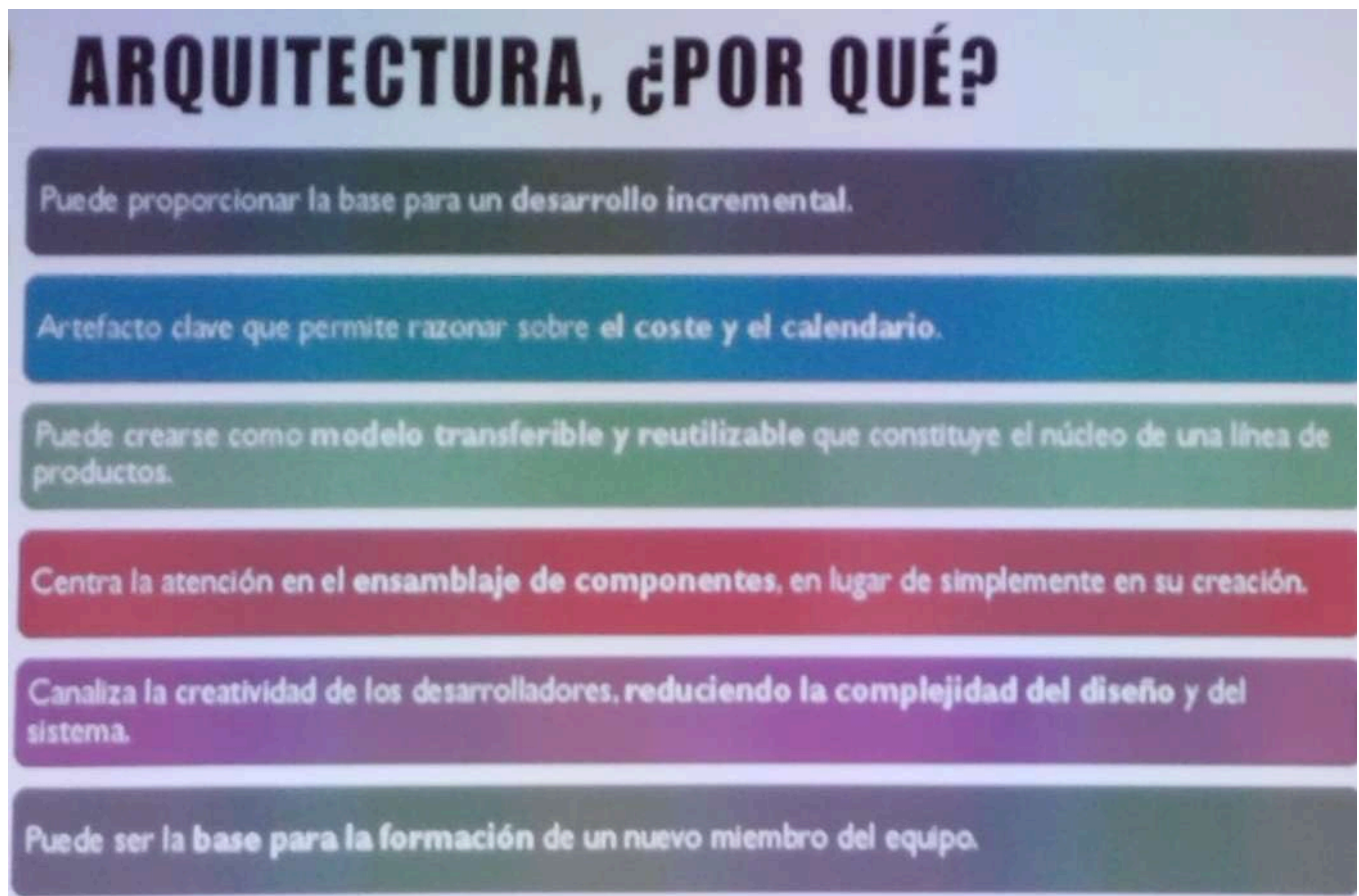
El objetivo de la arquitectura es simplificar el modelado para no tener que entrar en un nivel muy alto de detalle.

La arquitectura es diseño, pero no todo el diseño es arquitectura.

Todo sistema de software tiene una arquitectura de software. Esto es porque todos los sistemas tienen relaciones y elementos.

No todas las arquitecturas son buenas.

ARQUITECTURA, POR QUÉ?



(esto me dio chat gpt)

Base para un desarrollo incremental:

La arquitectura permite construir el sistema por partes, añadiendo funcionalidades gradualmente sin romper lo ya hecho.

Artefacto clave para razonar sobre coste y calendario:

Una buena arquitectura ayuda a estimar tiempos y costos de desarrollo más realistas, porque define claramente el alcance y la complejidad.

Modelo transferible y reutilizable:

La arquitectura puede servir como plantilla o referencia para otros proyectos similares, ahorrando tiempo y esfuerzo.

Foco en el ensamblaje de componentes:

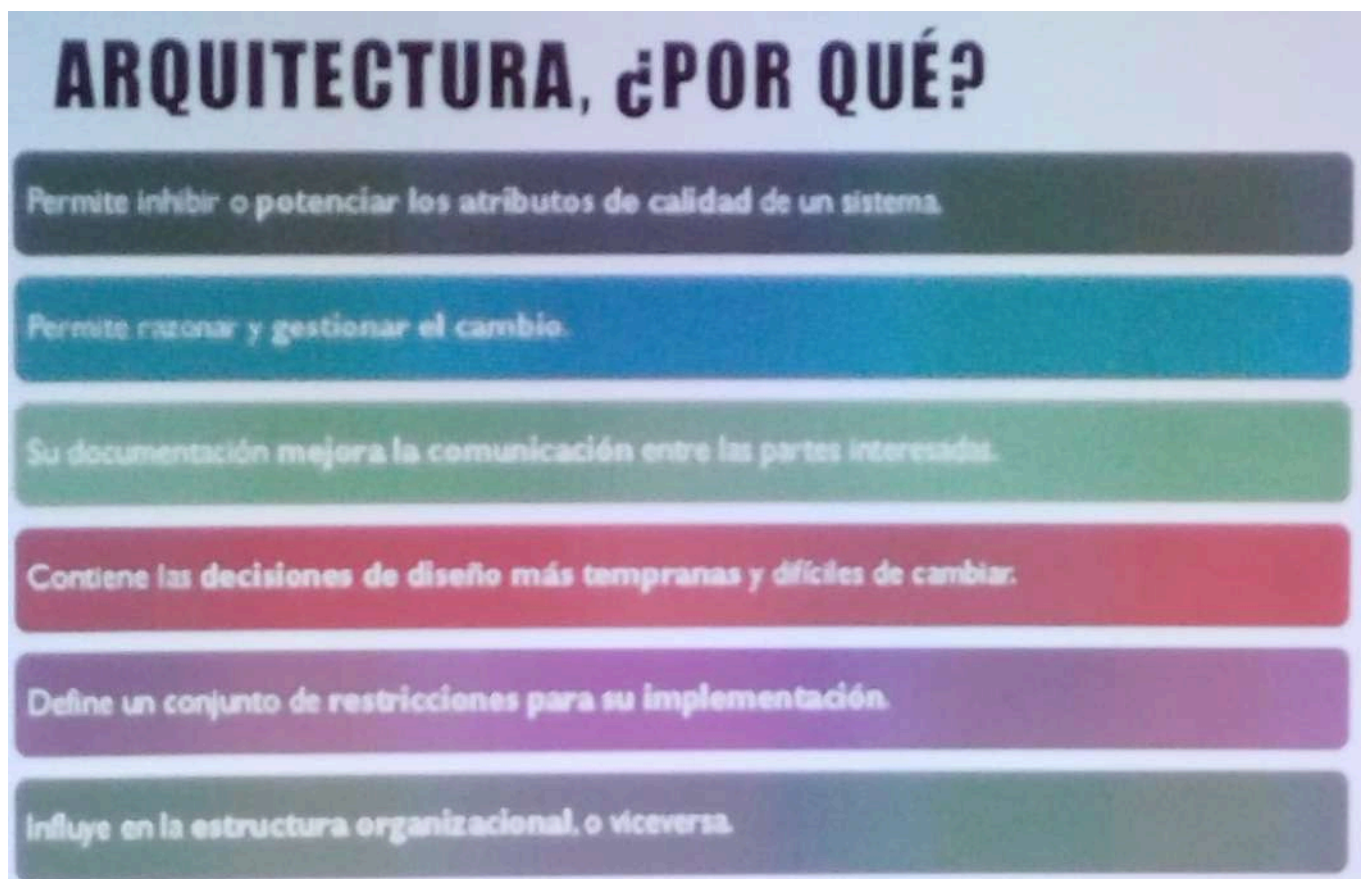
Facilita combinar módulos existentes en lugar de desarrollarlos desde cero, promoviendo la reutilización.

Reduce la complejidad del diseño y del sistema:

Al tener una estructura clara, los desarrolladores pueden centrarse en resolver problemas concretos sin preocuparse por el sistema completo.

Base para la formación de nuevos miembros:

Sirve como guía o mapa del sistema para que los nuevos integrantes comprendan rápidamente cómo está organizado el software.



(esto me dio chat gpt)

Permite potenciar o inhibir atributos de calidad:

La arquitectura influye en características como rendimiento, seguridad, escalabilidad o mantenibilidad.

Permite razonar y gestionar el cambio:

Una arquitectura bien pensada facilita hacer modificaciones sin afectar todo el sistema.

Mejora la comunicación entre las partes interesadas:

Su documentación actúa como lenguaje común entre desarrolladores, clientes y gestores.

Contiene decisiones de diseño tempranas y difíciles de cambiar:

Las primeras decisiones arquitectónicas impactan fuertemente el proyecto, por lo que deben tomarse con cuidado.

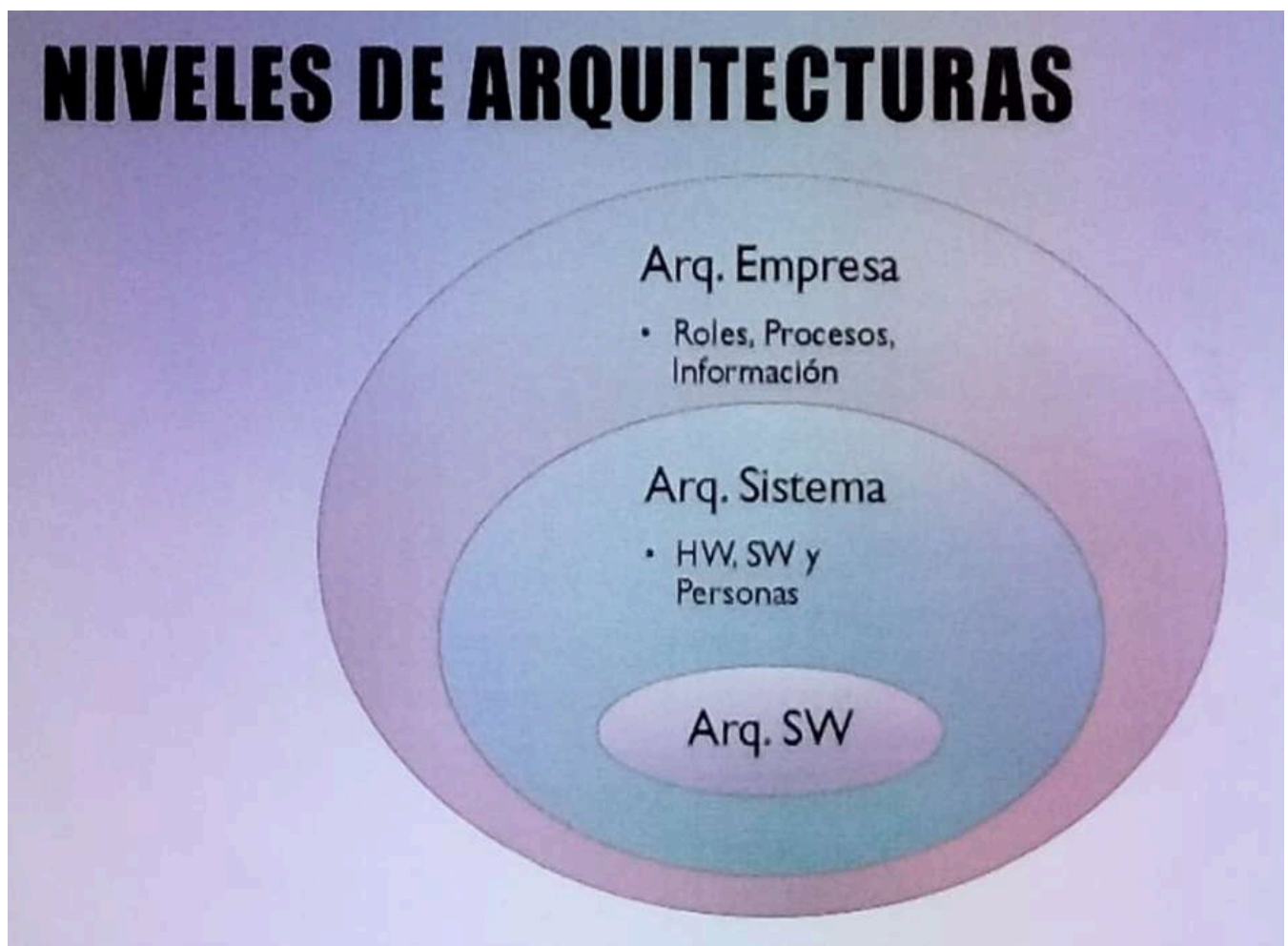
Define restricciones de implementación:

Establece límites técnicos (por ejemplo, lenguajes, frameworks, estilos de comunicación) que guían el desarrollo.

Influye en la estructura organizacional (o viceversa):

La forma en que está organizada la arquitectura suele reflejar cómo está estructurado el equipo (ley de Conway).

NIVELES DE ARQUITECTURAS



-Dentro de todo lo que abarca la arquitectura, la Arquitectura de Software es el nivel más pequeño y específico. Que se enfoca en la parte lógica vamos a decir

- no es lo mismo que arq. sistema, porque se tiene en cuenta el hw(incluyendo redes y demás) y las personas
- es muy difícil quedarse ahí, porque hay arquitecturas que deben referirse al hw
- en arquitectura de sistema va otra abstracción, no va a tanto nivel de detalle

-la Arquitectura de Empresa está por encima de ambas: se ocupa de los roles, procesos y la información que sostienen el funcionamiento global de la organización.

(hoy en día se utiliza mucho un framework para tratar la arquitectura empresarial)

ESTR. DE COMPONENTES Y CONECTORES

ESTR. DE COMPONENTES Y CONECTORES

- Se centra en la forma en que los elementos interactúan entre sí en tiempo de ejecución para llevar a cabo las funciones del sistema.
- Los **componentes** son las unidades principales de cálculo (comportamiento en tiempo de ejecución),
 - servicios, peers, clientes, servidores, etc.
- Los **conectores** son los vehículos de comunicación entre componentes,
 - devolución de llamadas, operadores de sincronización de procesos, tuberías u otros.

1) casi ningún diagrama de lo que habíamos dado se refiere a tiempos de ejecución(el diagrama de artefactos se puede llegar a parecer dijo). Ahora ya se refiere a servicios, ejecutables, cosas que están corriendo en memoria pero sin decirme exactamente su ubicación física en un nodo

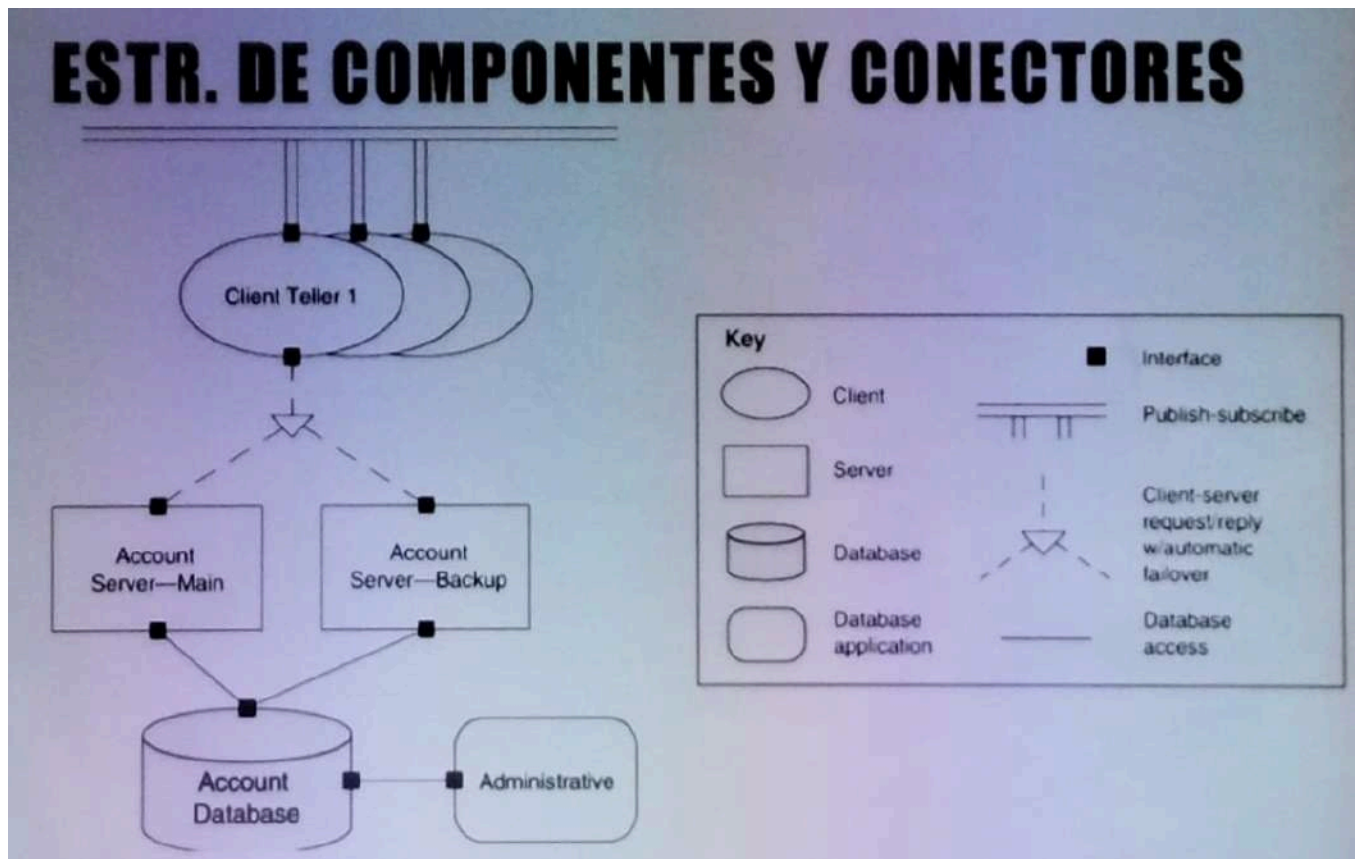
-cuando se habla de componentes no se habla de componente lógico, sino ya habla de componentes físicos. Los componentes pueden ser servicios, puertos, clientes, servidores, filtros o cualquier tipo de elemento en tiempo de ejecución.

-la definición de conectores tmb cambia

del Libro:

Este tipo de estructura es CRUCIAL para preguntarse propiedades de ejecución del sistema como tiempo, velocidad, seguridad, disponibilidad, etc.

EJEMPLO



- no se asemeja nada a UML, se referencia que significa cada cosa igualmente
- es un patrón de cliente-servidor

- los servidores se comunican con base de datos, la base de datos tiene una aplicación de gestión, los clientes se pueden comunicar entre ellos
- (el cliente se podría interpretar como un componente. Como me interesa las relaciones, me centro en la dinámica)
- hay dos servidores, uno principal y uno de backup
- si falla el principal va ir al backup
- no dice si la base de datos está replicada o no
- solo muestra la parte de ejecución

ESTRUCTURAS DE MÓDULOS

ESTRUCTURAS DE MÓDULOS

- Muestra cómo se estructura un sistema como un conjunto de unidades de código o datos que deben construirse o adquirirse.
- Los **módulos** representan una forma **estática** de considerar el sistema. Se les asignan áreas de responsabilidad funcional (menor énfasis en el tiempo de ejecución).
- Las implementaciones incluyen **paquetes, clases y capas**.
- Las **relaciones** entre módulos incluyen: **usa, generalización y forma parte de**.

-voy a tener equipos de trabajo, donde cada uno se va a especializar en hacer uno o otro módulo

-esto es estático

4) típicas construcciones que se pueden mapear en la estructuras de módulos

-de los conceptos de UML, este es el que más se parece

(son autores distintos, es un material diferente. Está hablando de arquitectura)

Del libro:

Llamamos módulos a las particiones del sistema que son implementadas. Representan una forma estática de modelar el sistema.

Nosotros conocemos a los elementos de una estructura de módulos como clases, paquetes, capas o meras divisiones de funcionalidad (todas son unidades de implementación).

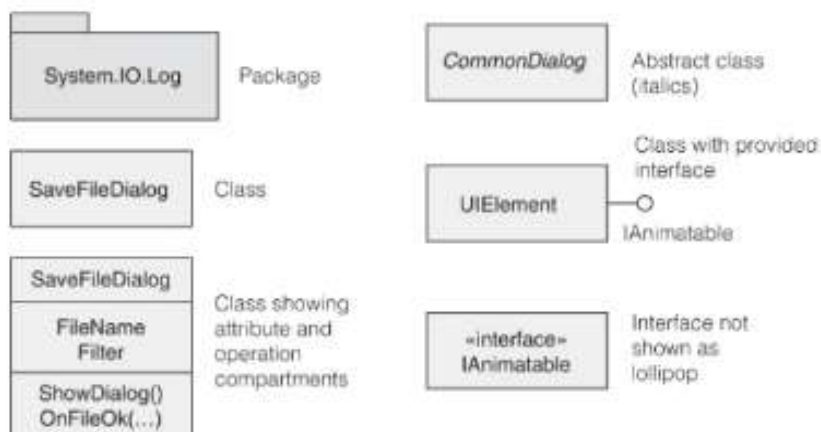


Figure 1.3 Module elements in UML

Las estructuras de módulo nos permiten preguntarnos:
Cuál es la funcionalidad principal asignada a cada módulo?
Que otros elementos de software son módulos permitidos para usar?
Que otro software usa este módulo y depende de él?
Que módulos están relacionados y qué tipo de relación hay entre ellos?

EJEMPLOS

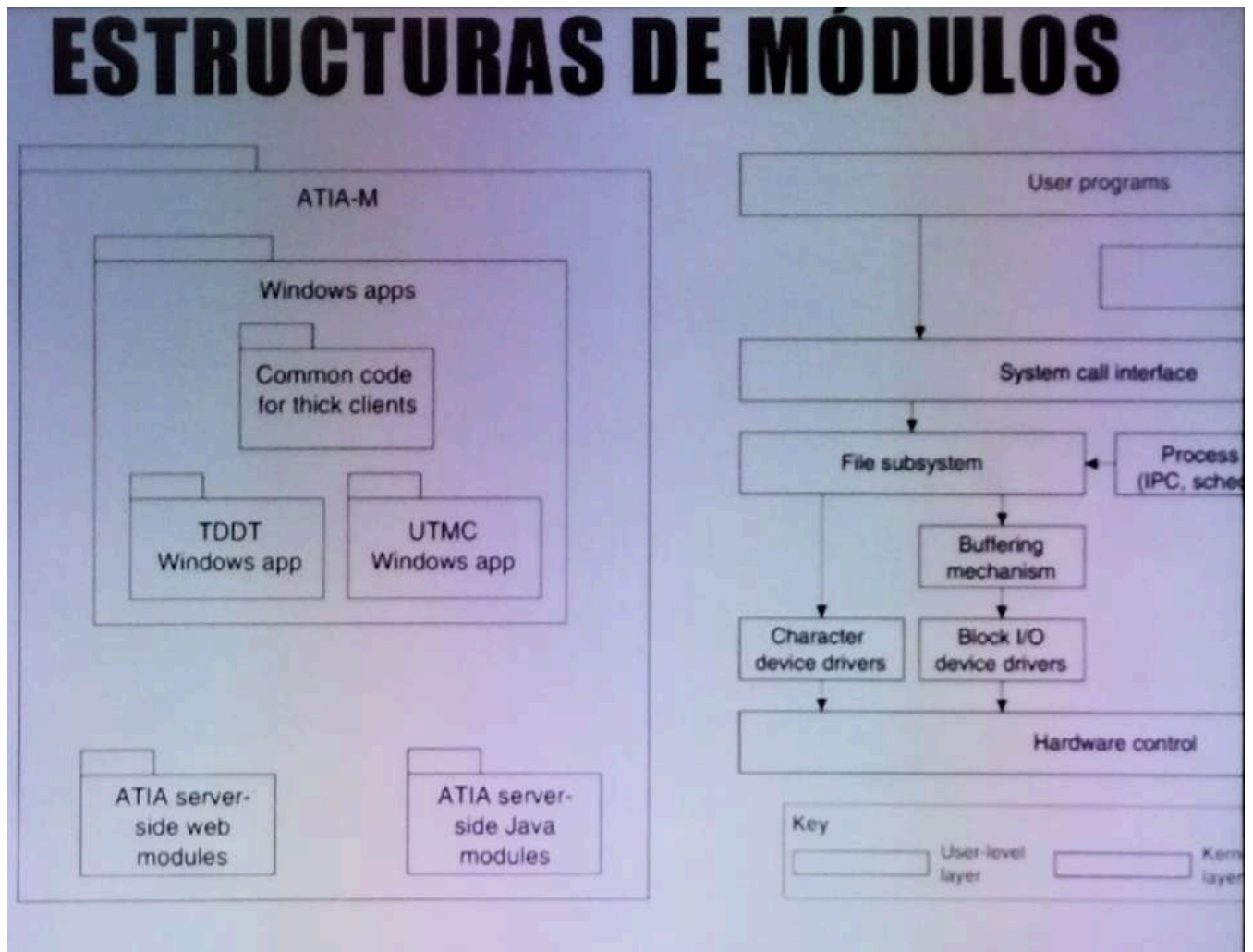


Figure 1.5 A decomposition structure

(es el mismo pero saqué del libro así está completo)

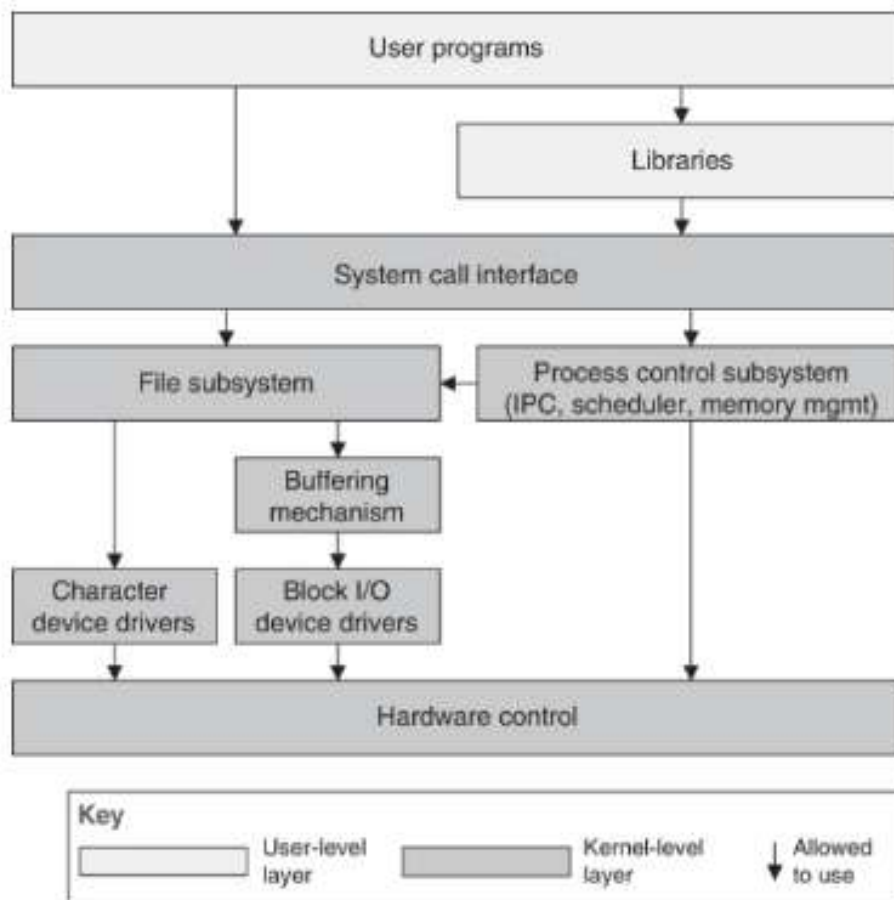


Figure 1.7 Layer structure

ESTRUCTURAS DE ASIGNACIÓN

ESTRUCTURAS DE ASIGNACIÓN

- Establecen la **asignación de los elementos de SW (C&C y módulos) a las estructuras de “no-SW”** del sistema (organización o sus entornos de desarrollo, prueba y ejecución)
 - Despliegue: asignación de SW (ejecución) a HW (o HW virtualizado)
 - Implementación: asignación de SW (módulos) a estructuras en los entornos de control de desarrollo, integración, prueba o configuración del sistema
 - Trabajo: asignación de SW a equipos de trabajo

- (típicamente se pueden representar muy bien con diagramas de despliegue UML)
- nos permite modelar que elementos del sw se asignan a otros elementos que no son sw
- Implementación: no solo es hw, podemos asignar módulos a otras variantes.

EJEMPLO

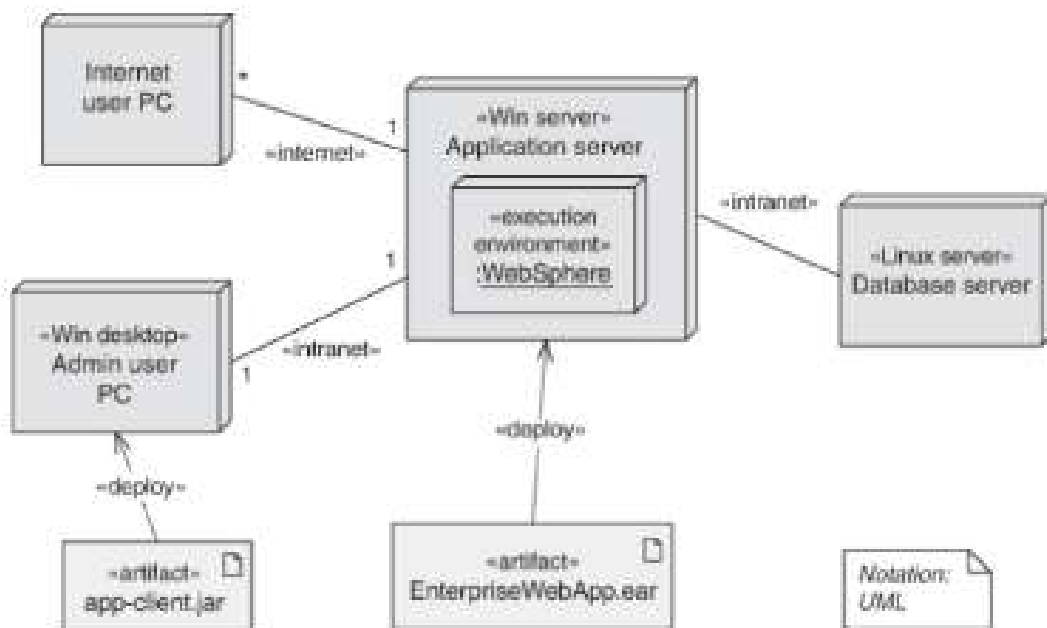
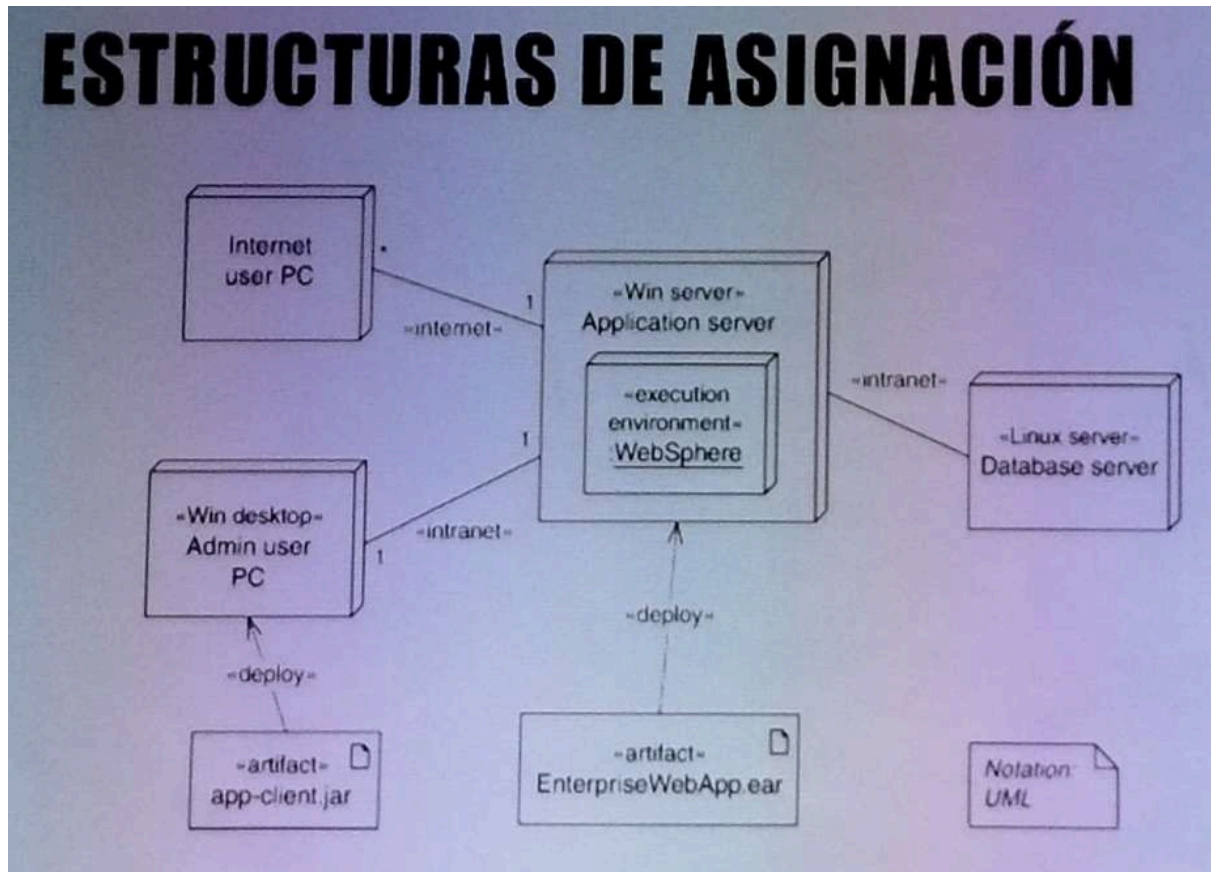


Figure 1.10 Deployment structure

ATRIBUTOS DE CALIDAD

ATRIBUTOS DE CALIDAD

- Propiedad medible o comprobable que refleja qué tan bien satisface el sistema las necesidades de los stakeholders más allá de la funcionalidad básica.
- La arquitectura es (generalmente) el primer lugar donde los atributos de calidad se consideran.
- Muchos sistemas se rediseñan por problemas de mantenibilidad, seguridad, escalabilidad, rendimiento, etc. y no por funcionalidad.
- La **funcionalidad no determina la arquitectura**.
- La funcionalidad se logra mediante **descomposición en módulos**.
- La arquitectura **restringe la asignación de responsabilidades** según atributos de calidad.

-son propiedades diferentes a los que son las funcionalidades, es a lo que se presta atención cuando vamos a sw.

-habla que tan bien satisface las necesidades del stakeholder, debe ser medible.

-seamos precisos en que es lo que queremos lograr

4) Los atributos funcionales no definen como tal la arquitectura solamente ayudan. La forma de la arquitectura te lo determina los requisitos no funcionales.

-esa distribución de responsabilidades se dan de acuerdo a como deben satisfacer las necesidades

-en un sw monolítico es difícil de mantener

Del libro:

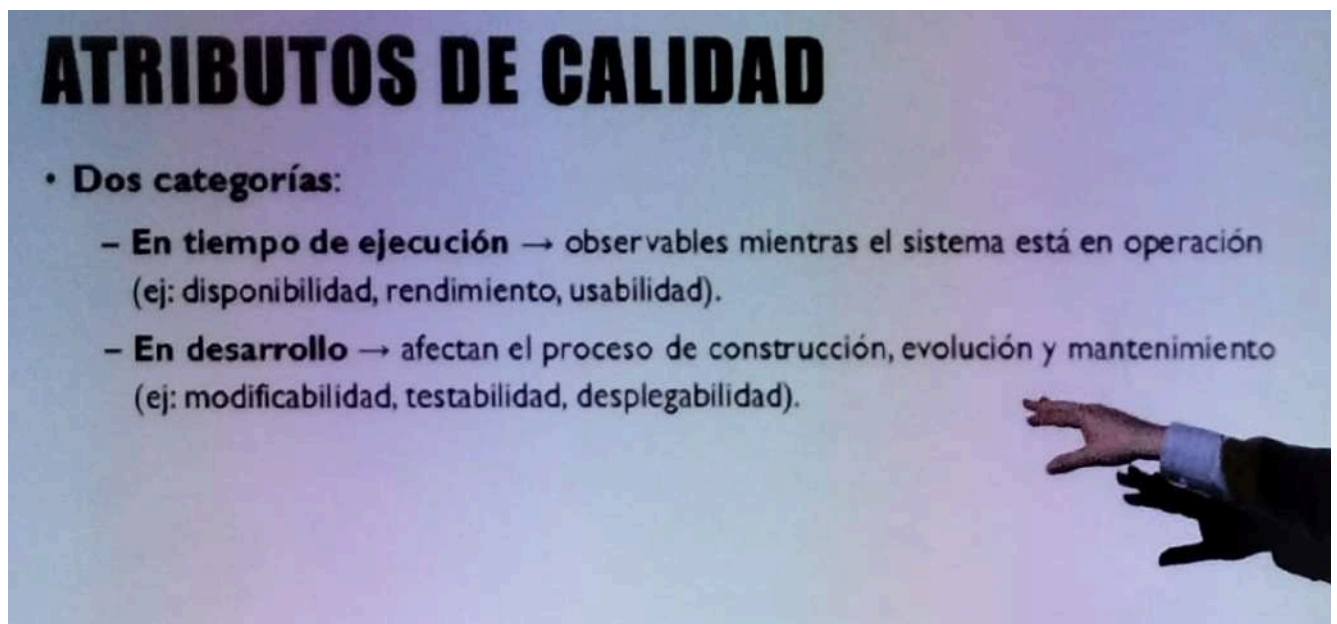
-La arquitectura se enfoca en los atributos NO funcionales más que en la funcionalidad.

-La habilidad de un sistema de aplicar sus atributos de calidad requeridos es determinado sustancialmente por su arquitectura.

-Aunque la funcionalidad es independiente de cualquier estructura, se llega a ella asignando responsabilidades a los elementos. Este proceso se denomina Descomposición de Módulos.

-El interés del arquitecto con respecto a la funcionalidad es en cómo interactúa y restringe a las demás cualidades.

Categorías



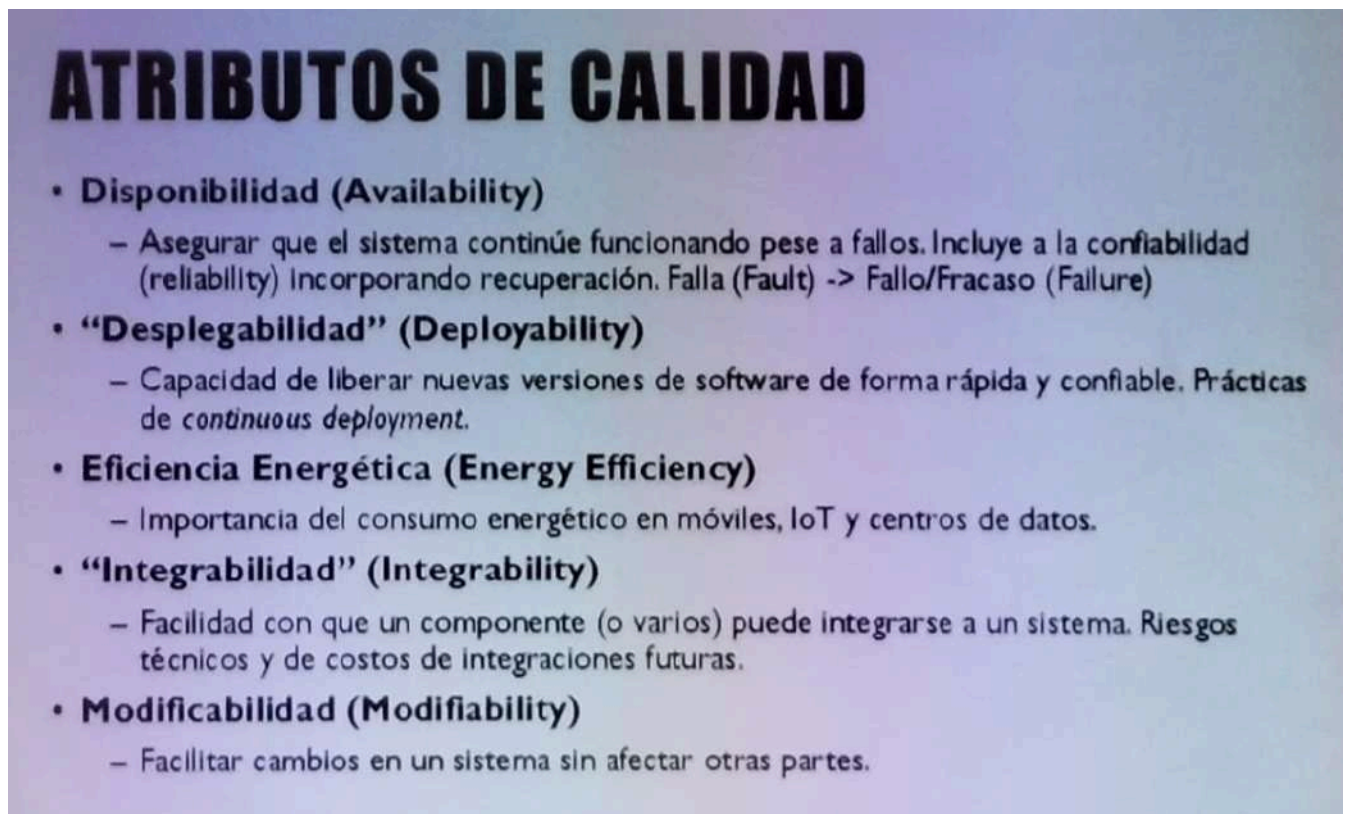
ATRIBUTOS DE CALIDAD

- **Dos categorías:**
 - **En tiempo de ejecución** → observables mientras el sistema está en operación (ej: disponibilidad, rendimiento, usabilidad).
 - **En desarrollo** → afectan el proceso de construcción, evolución y mantenimiento (ej: modificabilidad, testabilidad, despleabilidad).

A hand is visible on the right side of the slide, pointing towards the text.

-la mas importante tiene que ver con la de modificabilidad

Ejemplos



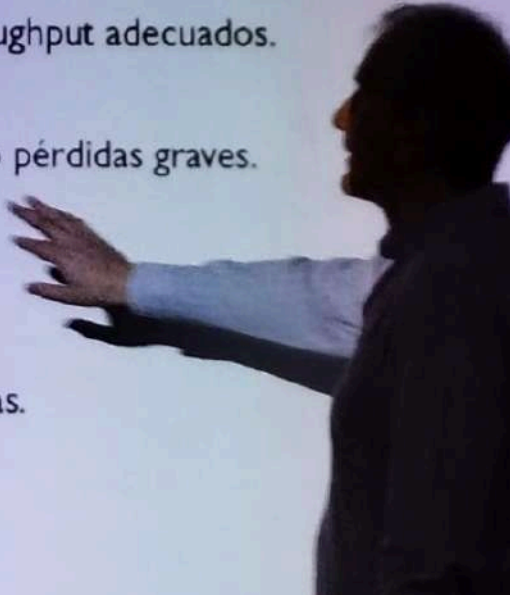
ATRIBUTOS DE CALIDAD

- **Disponibilidad (Availability)**
 - Asegurar que el sistema continúe funcionando pese a fallos. Incluye a la confiabilidad (reliability) Incorporando recuperación. Falla (Fault) -> Fallo/Fracaso (Failure)
- **“Desplegabilidad” (Deployability)**
 - Capacidad de liberar nuevas versiones de software de forma rápida y confiable. Prácticas de *continuous deployment*.
- **Eficiencia Energética (Energy Efficiency)**
 - Importancia del consumo energético en móviles, IoT y centros de datos.
- **“Integrabilidad” (Integrability)**
 - Facilidad con que un componente (o varios) puede integrarse a un sistema. Riesgos técnicos y de costos de integraciones futuras.
- **Modificabilidad (Modifiability)**
 - Facilitar cambios en un sistema sin afectar otras partes.

(los que están entre “” es porque la traducción en español no existe)

3) este fue apareciendo en los últimos tiempos

ATRIBUTOS DE CALIDAD

- **Rendimiento (Performance)**
 - Garantizar tiempos de respuesta y throughput adecuados.
 - **Fiabilidad física (Safety)**
 - Evitar que el sistema cause daño físico o pérdidas graves.
 - **Seguridad (Security)**
 - Proteger sistemas frente a ataques.
 - **Testeabilidad (Testability)**
 - Diseñar sistemas que faciliten las pruebas.
 - **Usabilidad (Usability)**
 - Facilidad de uso para el usuario final.
- 
- A person is shown from the side, pointing their right hand towards the text on the slide. They are wearing a light-colored shirt and a dark jacket.

2) Safety tiene que ver más con Confiabilidad

-la norma ISO 25010, tiene más de 30 atributos de calidad. Aca vemos lo más importantes

-todos estos deben permitir que esto se haga rápido y de manera confiable

-hay atributos que están interrelacionados, para poder desplegar sw debe ser testable fácilmente o rápidamente

ESCENARIOS ESPECIFICACIÓN DE AC

ESCENARIOS ESPECIFICACIÓN DE AC

- Forma de caracterizar los atributos de calidad, pues:
- Problemas comunes en la discusión de atributos:
 - **Definiciones no testables** (ej: “el sistema será modificable”).
 - **Solapamiento de categorías** (ej: un ataque DoS puede ser disponibilidad, seguridad, rendimiento o usabilidad).
 - **Vocabulario inconsistente** entre comunidades (eventos, ataques, fallas, entradas de usuario).
- Los escenarios tienen 6 partes...

(lo que veníamos viendo hablaba de forma de introducción, lo que dice el libro es)

2)

-que sea modificable no me dice nada, que significa eso?

-hay que tratar de ser específico cuando nos referimos a los atributos de calidad

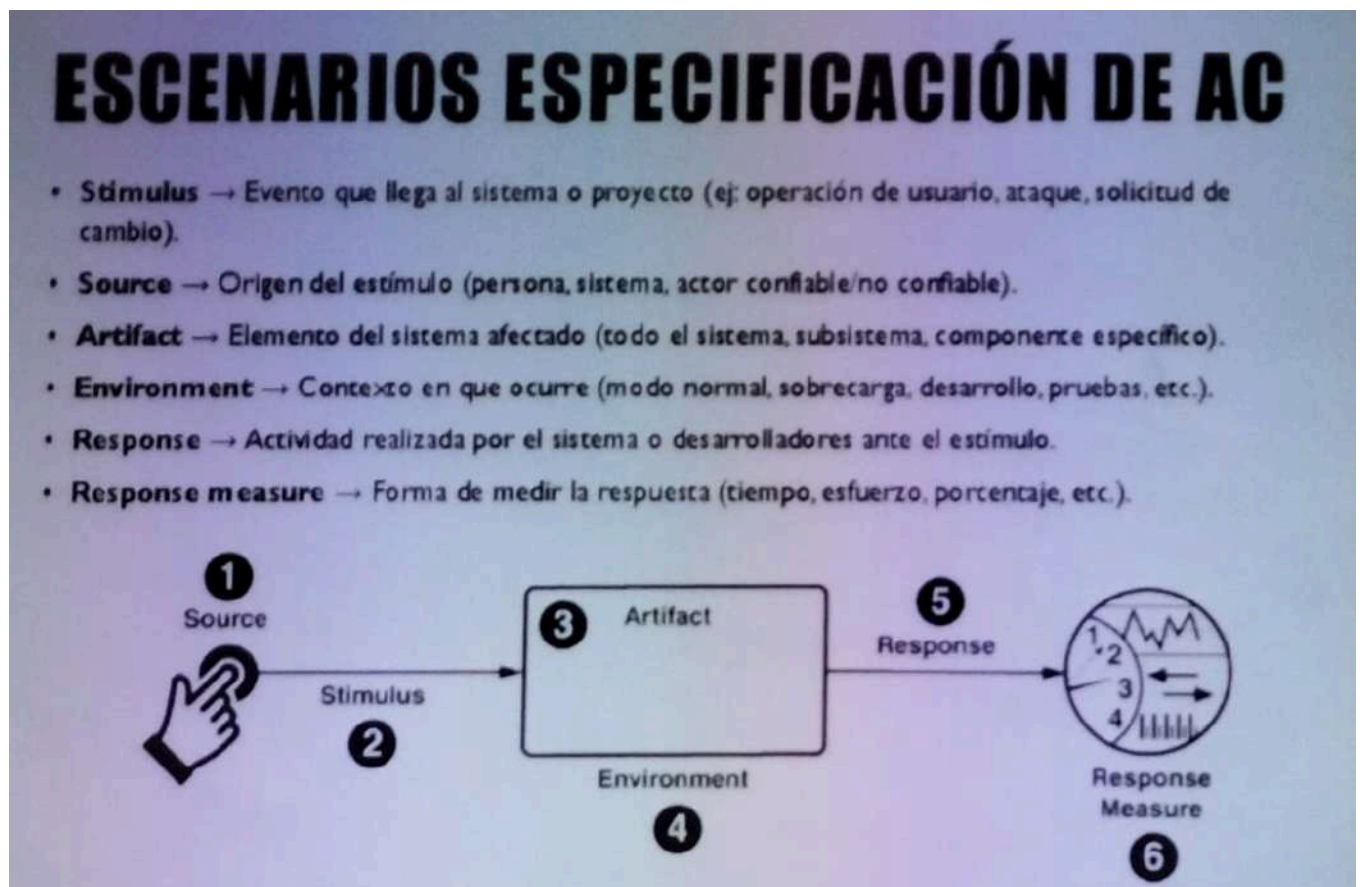


Figure 3.1 *The parts of a quality attribute scenario*

- el objetivo es evaluar la arquitectura

2) tiene una fuente de donde proviene el estímulo la falla o lo que sea

3) un artefacto en el que va a impactar, sistema completo, organización o componente.

4) Puede estar funcionando normalmente o a un sistema que ya tiene fallas. Puede ser uno en el que el sistema ni siquiera esté andando.

El estímulo lo recibe algún artefacto. Es obvio que viene al sistema pero es mejor ser preciso, ya que no es lo mismo cambiar algo de la interfaz que de la lógica de respuesta o cálculo.

5) cómo va a responder el sistema ante una falla. El estímulo se procesa y se debe generar una respuesta según las responsabilidades asignadas en el sistema.

6) la medición, si yo no puedo medir. No puedo saber si la decisión que tome es correcta

capítulo del 4 al 14 habla de cada atributo de calidad, antes habla de forma de introducción hay dos que nos vamos a enfocar, va a ser los que va a tomar

PATRONES Y TÁCTICAS

PATRONES Y TÁCTICAS

- **Tácticas** → decisiones de diseño que influyen en cómo el sistema responde a un estímulo (ej: mejorar portabilidad, rendimiento, integrabilidad). Afectan a un AC.
- **Patrones arquitectónicos** → soluciones probadas a problemas recurrentes; agrupan varios elementos, responsabilidades y relaciones. Afectan a más de un AC.
- Los patrones suelen incluir múltiples tácticas.
- Razones para enfocarse en tácticas:
 - Permiten **adaptar patrones** a contextos específicos.
 - Sirven para crear soluciones **cuando no hay un patrón existente**.
 - Hacen el **diseño y análisis más sistemático**.

-**Tácticas**: son formas o decisiones de diseño que interactúan o moldean el sistema, define como el sistema responde a un estímulo

-**Patrones arquitectónicos**: solución probada a problemas recurrentes, incluyen más de una táctica normalmente. Y generalmente resuelven atributos de calidad.

-no existen patrones para todos, así que a veces se puede combinar tácticas que no están en ningún patrón

Del libro:

No estamos inventando tácticas, sino simplemente plasmando las buenas prácticas de los arquitectos.

¿Por qué nos centramos en la táctica? Hay tres razones:

- 1) Los arquitectos frecuentemente necesitan modificar y adaptar los patrones a su contexto particular, y las tácticas proporcionan un medio sistemático para ampliar un patrón existente y subsanar todas sus deficiencias.
- 2) Si no existe un patrón para lograr el objetivo de diseño del arquitecto, las tácticas le permiten construir un fragmento de diseño a partir de "principios fundamentales".
- 3) Las tácticas ofrecen una forma de sistematizar el diseño y el análisis, dentro de ciertas limitaciones.

MODIFICABILIDAD

MODIFICABILIDAD

- Es la capacidad de un sistema para cambiarse o adaptarse fácilmente. Reduciendo costos y riesgos.
- Los cambios pueden ser para agregar o retirar funciones, corregir defectos, mejorar rendimiento o seguridad, actualizar tecnologías y estándares, o mejorar la experiencia del usuario.
- El arquitecto de preguntarse:
 - **¿Qué puede cambiar?**
Funciones, plataforma, entorno, protocolos, cualidades (rendimiento, confiabilidad) o capacidad del sistema.
 - **¿Qué probabilidad tiene el cambio?**
No se puede soportar todo tipo de cambio; se debe priorizar según lo más probable.
 - **¿Cuándo y quién hace el cambio?**
Puede ocurrir en diferentes fases (código, compilación, configuración, ejecución) y ser realizado por distintos actores (desarrolladores, usuarios, administradores o incluso el propio sistema en entornos adaptativos).
 - **¿Cuál es el costo del cambio?**

-menor tiempo, costo y riesgo posible

-un concepto muy parecido es el de mantenibilidad

-en esencia se trata de manejar los diferentes cambios, cualquiera que pueda existir.

-al principio tiene que realizarse estas preguntas

-hay que pensar en los que tienen mayor potencialidad de cambios. pensando que parte de la arquitectura tenga más probabilidad de cambiar

-potencialmente todo dentro de un sistema tiene posibles cambios, pero hay que concentrarse en lo que tenga mayor potencialidad

Del libro:

Ahora, sin embargo, la cuestión de cuándo se realiza un cambio está intrínsecamente ligada a la cuestión de quién lo realiza.

¿Cuándo y quién hace el cambio?

Se pueden realizar distintos tipos de cambio:

Implementación (Modificando código fuente).

En tiempo de compilación.

Durante la construcción (mediante la selección de bibliotecas).

Durante la configuración de las opciones (Mediante diversas técnicas, incluyendo la configuración de parámetros).

En tiempo de ejecución (mediante la configuración de parámetros, complementos, asignación de hardware, etc.).

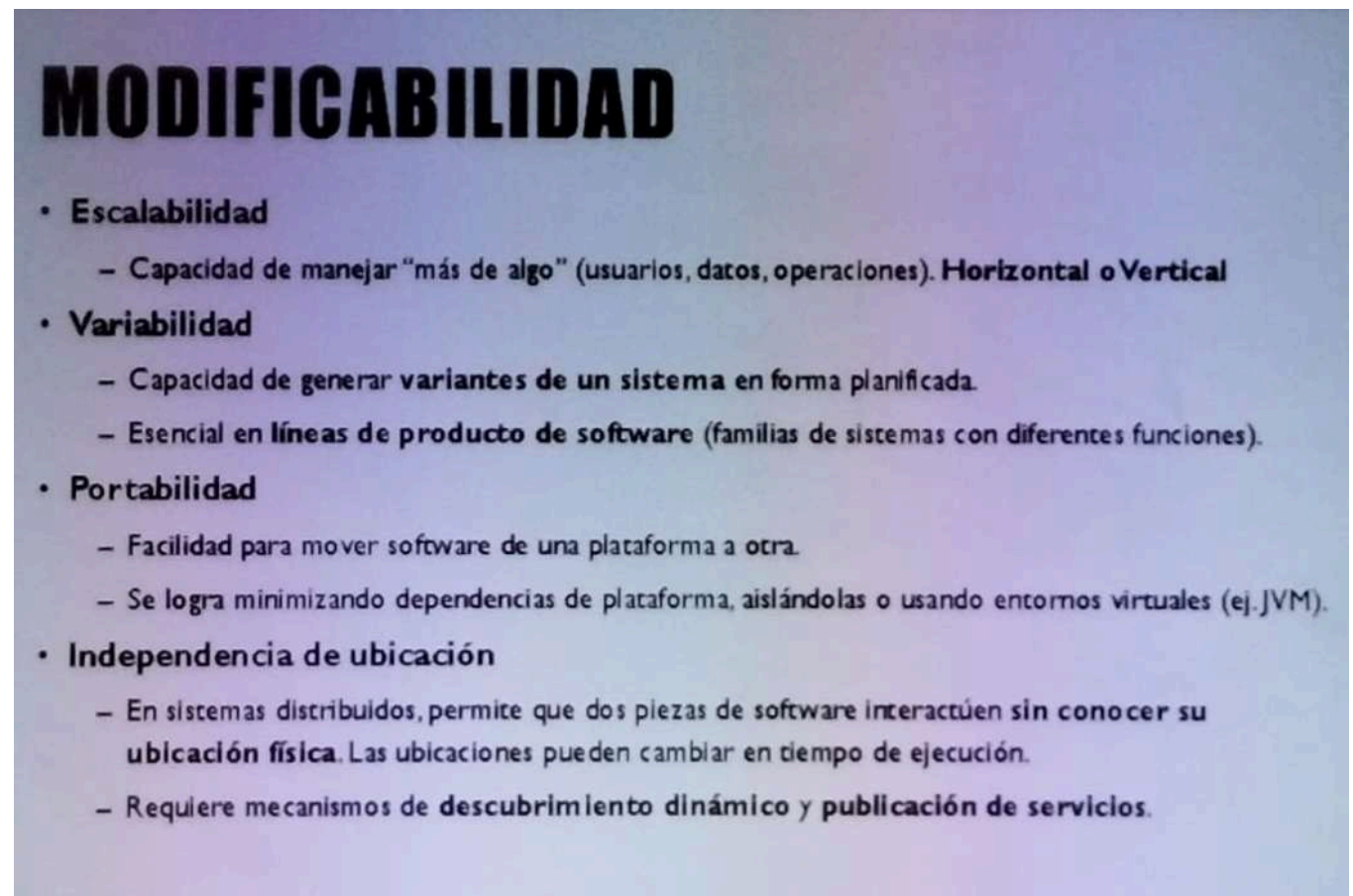
Cuánto cuesta hacer el cambio? Hay 2 costos de modificabilidad:

El costo de introducir el mecanismo de cambio para hacer el sistema más modificable

El costo de hacer la modificación utilizando ese mecanismo.

No se debería realizar cambio tras cambio de la misma cosa, sino realizar un mecanismo arquitectural para evitar este problema y no contraer deuda técnica.

Tipos de Modificabilidad



MODIFICABILIDAD

- **Escalabilidad**
 - Capacidad de manejar “más de algo” (usuarios, datos, operaciones). **Horizontal o Vertical**
- **Variabilidad**
 - Capacidad de generar **variantes de un sistema** en forma planificada.
 - Esencial en **líneas de producto de software** (familias de sistemas con diferentes funciones).
- **Portabilidad**
 - Facilidad para mover software de una plataforma a otra.
 - Se logra minimizando dependencias de plataforma, aislándolas o usando entornos virtuales (ej. JVM).
- **Independencia de ubicación**
 - En sistemas distribuidos, permite que dos piezas de software interactúen **sin conocer su ubicación física**. Las ubicaciones pueden cambiar en tiempo de ejecución.
 - Requiere mecanismos de **descubrimiento dinámico y publicación de servicios**.

Escalabilidad -esta entre la modificabilidad y performance. habla de manejar más de algo desde el punto de vista del sw. Tipos de escalabilidad

Del libro:

Escalabilidad: se trata de añadir más recursos a lo que ya hay. Existe

-Horizontal (afuera): Se añaden más recursos a unidades lógicas como más clusters para servidores. En sistemas basados en la nube a este tipo se le llama Elasticidad.

-Vertical (arriba): Se añaden más recursos físicos como más memoria a una computadora.

El problema que surge con ambos tipos de escalado es cómo utilizar eficazmente los recursos adicionales.

Ser eficaz significa que los recursos adicionales resultan en una mejora medible de la calidad del sistema, no requieren un esfuerzo excesivo para su implementación y no interrumpen indebidamente las operaciones.

Variabilidad: es la capacidad de un sistema y de todos sus artefactos asociados (código, requisitos, documentación, planes de prueba, etc.) para permitir la producción de distintas variantes del producto de manera planificada. Este atributo es especialmente importante en una línea de productos de software, donde existe una familia de sistemas similares que difieren en características o funciones.

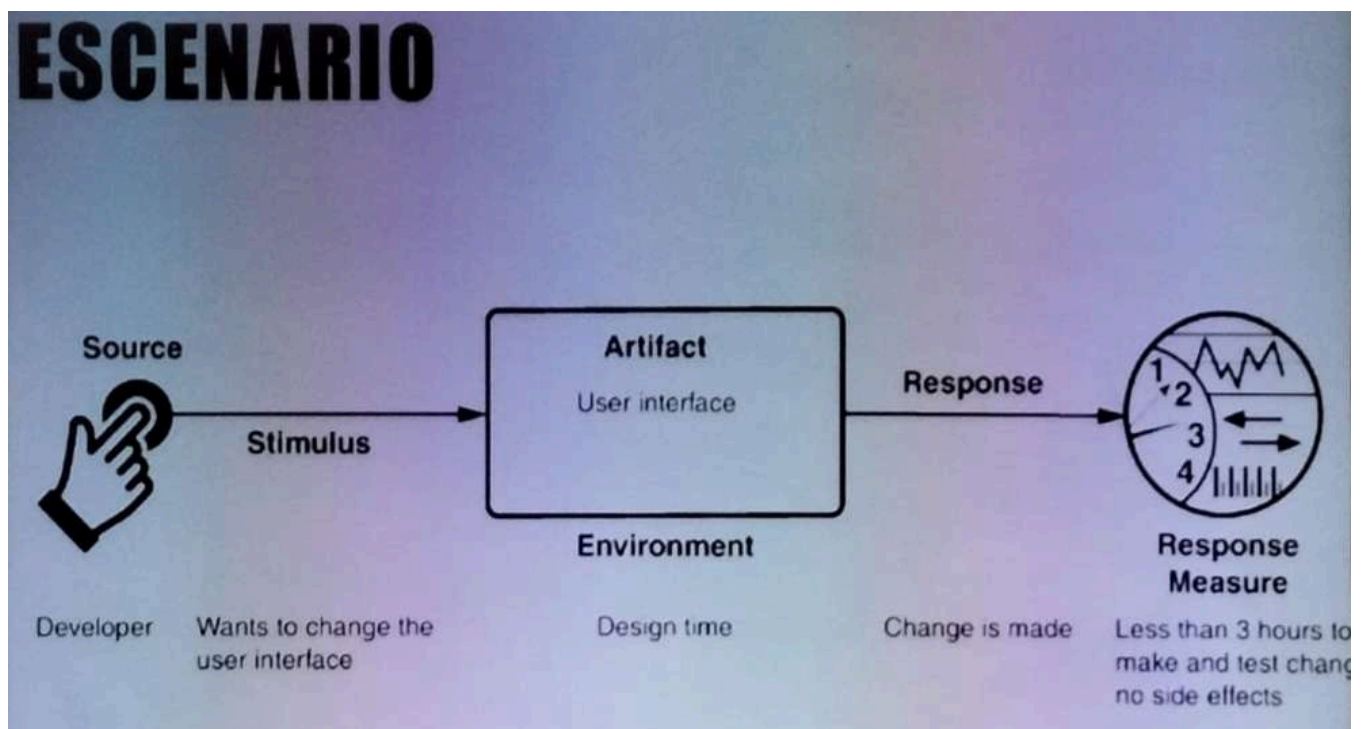
Cuando los artefactos de ingeniería pueden compartirse entre los distintos miembros de la familia, el costo general de producir nuevas variantes disminuye drásticamente. Esto se logra mediante mecanismos que permiten seleccionar o adaptar artefactos según los distintos contextos de uso contemplados en la línea de productos. El objetivo principal de la variabilidad es facilitar la construcción y el mantenimiento de los productos de esa familia a lo largo del tiempo.

Portabilidad: se refiere a la facilidad con la que un software diseñado para ejecutarse en una plataforma puede adaptarse para ejecutarse en otra.

La portabilidad se logra minimizando las dependencias de plataforma del software, aislando dichas dependencias en ubicaciones bien identificadas y escribiendo el software para que se ejecute en una máquina virtual que encapsula todas las dependencias de plataforma.

Independencia de ubicación: se refiere al caso en el que dos piezas de software distribuido interactúan sin conocer en tiempo de ejecución dónde están. Además, su ubicación incluso podría cambiar. Si se diseña un sistema con independencia de ubicación, significa que este podrá modificar la ubicación del mismo sin causar mucho impacto en el resto del sistema.

Escenario



-lo interesante de acá es que me va a definir que un sistema es modificable, que lo va hacer de forma masomenos rápida

-puede llegar a pasar que el cambio no se pueda hacer, eso es aún peor

-se considera aprobado si las pruebas fueron exitosas, y por debajo del tiempo indicado

TÁCTICAS

- Incrementar Cohesión
 - Dividir módulo
 - Redistribuir responsabilidades
- Reducir Acoplamiento
 - Encapsular
 - Usar un intermediario
 - Abstraer servicios comunes (logging, bases de datos)
 - Restringir dependencias
- Diferir Enlace
 - Enlazar lo más tarde posible, siempre que el mecanismo que lo permita sea rentable. Tiempo de compilación, de construcción, despliegue, inicialización y ejecución.

(habla de los items principales, dps hay que leer el resto)

3) significa poder cambiar mi sistema lo más cerca del tiempo de ejecución posible. Significa que es más modificable porque tengo menos cambios que hacer. Es el momento en el que yo puedo hacer los cambios.

Del libro:

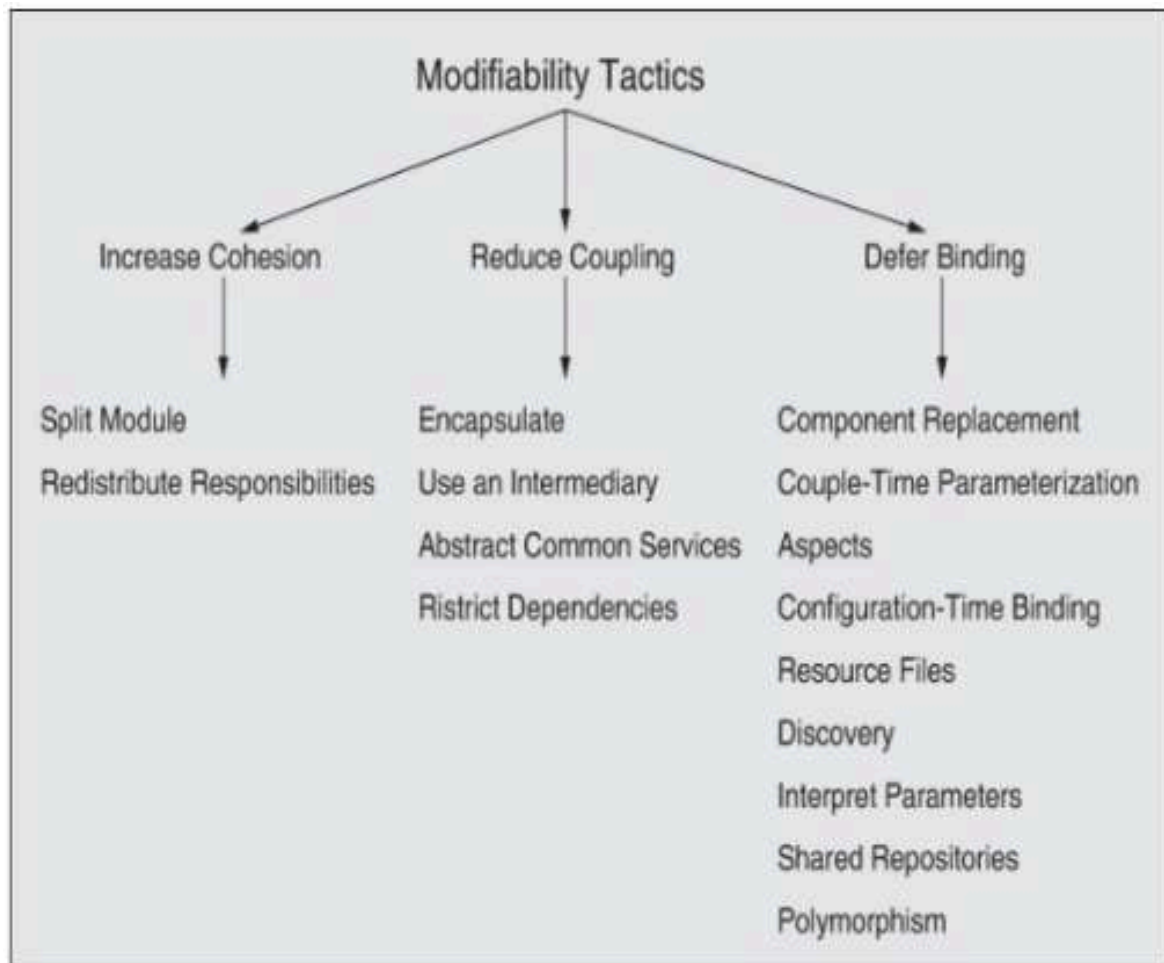
La cohesión mide la fuerza de la relación entre las responsabilidades de un módulo.

La cohesión de un módulo es la probabilidad de que un escenario de cambio que afecte a una responsabilidad también afecte a otras responsabilidades. Cuanto mayor sea la cohesión, menor será la probabilidad de que un cambio determinado afecte a varios módulos.

El tamaño del módulo también influye. Módulos más grandes son más difíciles de cambiar y tienen tendencia a tener errores.

Debemos considerar el punto del ciclo de vida del desarrollo de software en el que se produce un cambio. Si ignoramos el coste de preparar la arquitectura para la modificación, preferimos que un cambio se vincule lo más tarde posible. Una arquitectura adecuadamente equipada para incorporar modificaciones al final del ciclo de vida costará, en promedio, menos que una arquitectura que fuerce la misma modificación antes.

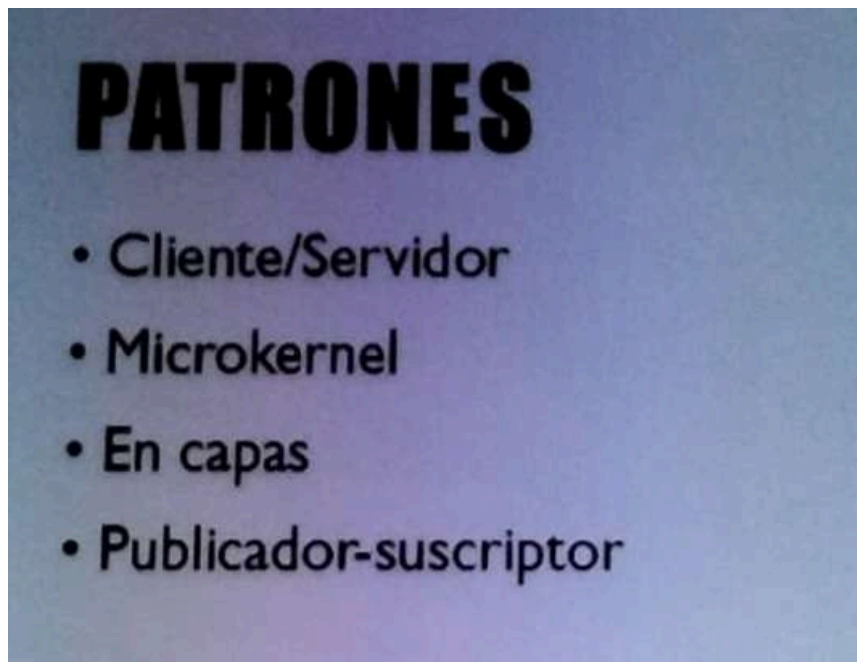
Diferir enlace: La preparación frente a modificaciones en etapas tardías del ciclo de vida del desarrollo de software también son buenas prácticas. Un sistema bien preparado puede reducir el costo a cero o bajarlo muchísimo.



Tácticas que plantea el libro y cuando pueden usarse

- **En tiempo de compilación/construcción:**
 - Reemplazo de componentes (ej. en un script de build)
 - Parametrización en tiempo de compilación
 - Aspectos
- **En tiempo de despliegue/arranque:**
 - Binding en tiempo de configuración
 - Archivos de recursos (ej. `.properties`, `.json`, `.env`)
- **En tiempo de ejecución:**
 - Descubrimiento (Discovery)
 - Interpretar parámetros
 - Repositorios compartidos
 - Polimorfismo

PATRONES



(no vamos a entrar en detalle, quiere que veamos la conexiones)

-estos son algunos patrones que me van a permitir que mi sistema sea más modificable

-es como que tiene armado un sistema para cuando yo me quiero enfocar en determinado atributo de calidad, me indica cuales son los patrones y tácticas posibles para ese atributo de calidad

Del libro:

Cliente/Servidor

El patrón cliente-servidor consiste en un servidor que proporciona servicios simultáneamente a múltiples clientes distribuidos. El ejemplo más común es un servidor web que proporciona información a múltiples usuarios simultáneos de un sitio web.

Las interacciones entre un servidor y sus clientes siguen esta secuencia:

- Descubrimiento:

- La comunicación la inicia un cliente, que utiliza un servicio de descubrimiento para determinar la ubicación del servidor.

- El servidor responde al cliente utilizando un protocolo acordado.

- Interacción:

- El cliente envía solicitudes al servidor.

- El servidor procesa las solicitudes y responde.

Cabe destacar varios puntos sobre esta secuencia:

- El servidor puede tener múltiples instancias si el número de clientes crece más allá de la capacidad de una sola instancia.

- Si el servidor no mantiene estado con respecto a los clientes, cada solicitud de un cliente se trata de forma independiente.

- Si el servidor mantiene estado con respecto a los clientes, entonces:

- Cada solicitud debe identificar al cliente de alguna manera.

- El cliente debe enviar un mensaje de «fin de sesión» para que el servidor pueda eliminar los recursos asociados a ese cliente.
- El servidor puede agotar el tiempo de espera si el cliente no ha enviado una solicitud en un tiempo determinado para que se puedan eliminar los recursos asociados al cliente.

Ventajas:

- La conexión entre un servidor y sus clientes se establece dinámicamente. El servidor no tiene conocimiento previo de sus clientes; es decir, existe un bajo acoplamiento entre el servidor y sus clientes.
- No existe acoplamiento entre los clientes.
- El número de clientes puede escalar fácilmente y solo está limitado por la capacidad del servidor. La funcionalidad del servidor también puede escalar si se excede su capacidad.
- Los clientes y servidores pueden evolucionar de forma independiente.
- Los servicios comunes pueden compartirse entre varios clientes.
- La interacción con un usuario se limita al cliente. Este factor ha dado lugar al desarrollo de lenguajes y herramientas especializadas para gestionar la interfaz de usuario.

Desventajas:

- Este patrón se implementa de forma que la comunicación se produce a través de una red, posiblemente incluso Internet. Por lo tanto, los mensajes pueden retrasarse debido a la congestión de la red, lo que conlleva una degradación (o al menos imprevisibilidad) del rendimiento.
- Para los clientes que se comunican con servidores a través de una red compartida por otras aplicaciones, deben adoptarse medidas especiales para lograr la seguridad (especialmente la confidencialidad) y mantener la integridad.

Plug-in (Microkernel)

El patrón de plug-ins tiene dos tipos de elementos: elementos que proporcionan un conjunto básico de funcionalidad y variantes especializadas (llamadas plug-ins) que añaden funcionalidad al núcleo mediante un conjunto fijo de interfaces. Los dos tipos se suelen vincular durante la compilación o posteriormente.

Algunos ejemplos de uso son los siguientes:

- La funcionalidad principal puede ser un sistema operativo reducido (el micronúcleo) que proporciona los mecanismos necesarios para implementar los servicios del sistema operativo, como la gestión del espacio de direcciones de bajo nivel, la gestión de hilos y la comunicación entre procesos (IPC). Los plug-ins proporcionan la funcionalidad real del sistema operativo, como los controladores de dispositivos, la gestión de tareas y la gestión de solicitudes de E/S.
- La funcionalidad principal es un producto que proporciona servicios a sus usuarios. Los plug-ins proporcionan portabilidad, como la compatibilidad con el sistema operativo o la compatibilidad con bibliotecas de soporte. Los plug-ins también pueden proporcionar funcionalidad adicional no incluida en el producto principal. Además, pueden actuar como adaptadores para permitir la integración con sistemas externos (véase el capítulo 7).

Beneficios:

- Los plug-ins proporcionan un mecanismo controlado para extender un producto principal y hacerlo útil en diversos contextos.
- Los plug-ins pueden ser desarrollados por equipos u organizaciones diferentes a los desarrolladores del micronúcleo. Esto permite el desarrollo de dos mercados distintos: uno para el producto principal y otro para los plug-ins.
- Los plug-ins pueden evolucionar independientemente del micronúcleo. Dado que interactúan a través de interfaces fijas, mientras estas no cambien, los dos tipos de elementos no están acoplados.

Desventajas:

- Debido a que los plug-ins pueden ser desarrollados por diferentes organizaciones, es más fácil introducir vulnerabilidades de seguridad y amenazas a la privacidad.

Layers (en capas)

El patrón de capas divide el sistema de tal manera que los módulos pueden desarrollarse y evolucionar por separado, con poca interacción entre las partes, lo que favorece la portabilidad, la modificabilidad y la reutilización. Para lograr esta separación de responsabilidades, el patrón de capas divide el software en unidades llamadas capas. Cada capa es una agrupación de módulos que ofrece un conjunto cohesivo de servicios. La relación de uso permitido entre las capas está sujeta a una restricción clave: las relaciones deben ser **unidireccionales**.

Las capas dividen completamente un conjunto de software, y cada partición se expone a través de una interfaz pública.

En algunos casos, se requiere que los módulos de una capa usen directamente módulos de una capa inferior no adyacente, aunque normalmente solo se permiten usos de la capa **inmediatamente inferior**. Este caso de software de una capa superior que usa módulos de una capa inferior no adyacente se denomina puente de capas(layer bridging). En este patrón, no se permiten usos ascendentes.

Beneficios:

- Dado que una capa está restringida a usar solo capas inferiores, el software en las capas inferiores **se puede modificar** (siempre que la interfaz no cambie) sin afectar las capas superiores.
- Las capas inferiores se pueden **reutilizar** en diferentes aplicaciones. Por ejemplo, supongamos que una determinada capa permite la portabilidad entre sistemas operativos. Esta capa sería útil en cualquier sistema que deba ejecutarse en múltiples sistemas operativos diferentes. Las capas más bajas suelen ser proporcionadas por software comercial, como un sistema operativo o software de comunicaciones de red.
- Debido a que las relaciones de uso permitidas están restringidas, se reduce la cantidad de interfaces que un equipo debe comprender.

Desventajas:

- Si la arquitectura en capas no está diseñada correctamente, puede resultar **contraproducente**, al no proporcionar las abstracciones de bajo nivel que los programadores de los niveles superiores necesitan.

- La arquitectura en capas suele penalizar el **rendimiento** del sistema. Si se realiza una llamada desde una función en la capa superior, esta puede tener que atravesar muchas capas inferiores antes de ser ejecutada por el hardware.
- Si se producen **muchos casos** de puenteo de capas, el sistema puede no cumplir con sus objetivos de portabilidad y modificación, que una arquitectura en capas estricta ayuda a lograr.

Publicador-suscriptor

El patrón de publicación-suscripción es un patrón arquitectónico en el que los componentes se comunican principalmente mediante mensajes asíncronos, a veces denominados «eventos» o «temas». Los publicadores desconocen a los suscriptores, y estos últimos solo conocen los tipos de mensajes. Los sistemas que utilizan el patrón de publicación-suscripción se basan en la invocación implícita; es decir, el componente que publica un mensaje no invoca directamente a ningún otro componente.

Los componentes publican mensajes en uno o más eventos o temas, y otros componentes registran su interés en la publicación. En tiempo de ejecución, cuando se publica un mensaje, el bus de publicación-suscripción (o de eventos) **notifica a todos** los elementos que registraron interés en el evento o tema. De esta manera, la publicación del mensaje provoca una invocación implícita de métodos en otros componentes. El resultado es un bajo acoplamiento entre los publicadores y los suscriptores.

Tiene tres tipos de elementos:

- Componente publicador. Envía (publica) mensajes.
- Componente suscriptor. Se suscribe a los mensajes y los recibe.
- Bus de eventos. Gestiona las suscripciones y el envío de mensajes como parte de la infraestructura de ejecución.

Ventajas:

- Los publicadores y suscriptores son independientes y, por lo tanto, están débilmente acoplados. Agregar o cambiar suscriptores solo requiere registrarse para un evento y no provoca cambios en el publicador.
- El **comportamiento del sistema** se puede modificar fácilmente cambiando el evento o el tema de un mensaje que se publica y, en consecuencia, qué suscriptores pueden recibirlo y actuar en consecuencia. Este cambio, aparentemente pequeño, puede tener grandes consecuencias, ya que se pueden activar o desactivar funciones al agregar o suprimir mensajes.
- Los eventos se pueden registrar fácilmente para permitir su grabación y reproducción, y así **reproducir condiciones de error** que pueden ser difíciles de recrear manualmente.

Desventajas:

- Algunas implementaciones del patrón de publicación-suscripción pueden afectar negativamente el **rendimiento** (latencia). El uso de un mecanismo de coordinación distribuida mitigará la degradación del rendimiento.
- En algunos casos, un componente no puede estar seguro de cuánto tiempo tardará en recibir un mensaje publicado. En general, el rendimiento del sistema y la gestión de recursos son más **difíciles de predecir** en sistemas de publicación-suscripción.

- El uso de este patrón puede afectar negativamente el **determinismo** de los sistemas síncronos. El orden en que se invocan los métodos, como resultado de un evento, puede variar en algunas implementaciones.
- El uso del patrón de publicación-suscripción puede afectar negativamente la capacidad de realizar **pruebas**. Cambios aparentemente pequeños en el bus de eventos, como un cambio en qué componentes están asociados con qué eventos, pueden tener un gran impacto en el comportamiento del sistema y la calidad del servicio.
- Algunas implementaciones de publicación-suscripción limitan los mecanismos disponibles para implementar la **seguridad** (integridad) de forma flexible. Dado que los editores desconocen la identidad de sus suscriptores, y viceversa, el cifrado de extremo a extremo es limitado. Los mensajes de un editor al bus de eventos pueden cifrarse de forma única, y los mensajes del bus de eventos a un suscriptor también pueden cifrarse de forma única; sin embargo, cualquier comunicación cifrada de extremo a extremo requiere que todos los editores y suscriptores involucrados compartan la misma clave.

PERFORMANCE

PERFORMANCE

- Capacidad del sistema para responder a eventos en un tiempo aceptable, cumpliendo con requisitos temporales.
- Fuentes de demora:
 - Computación → nanosegundos
 - Disco → milisegundos
 - Red → microsegundos a cientos de milisegundos
- Escenarios típicos:
 - Sistemas web: cantidad de solicitudes procesadas por unidad de tiempo.
 - Sistemas de control (ej. motor): variación permitida en tiempos críticos.

(esta en la categoría de tiempo de ejecución)

-es el rendimiento

-los atributos de calidad podrían funcionar en todo el sistema, es mejor enlazarlo a ciertas funcionalidades

Del libro:

Con rendimiento se refiere a tiempo, el cual debe tenerse en cuenta a la hora de diseñar el sistema y este tenga buen rendimiento.

Cuando ocurre algún evento o estímulo, el sistema debe responder a él en un tiempo razonable.

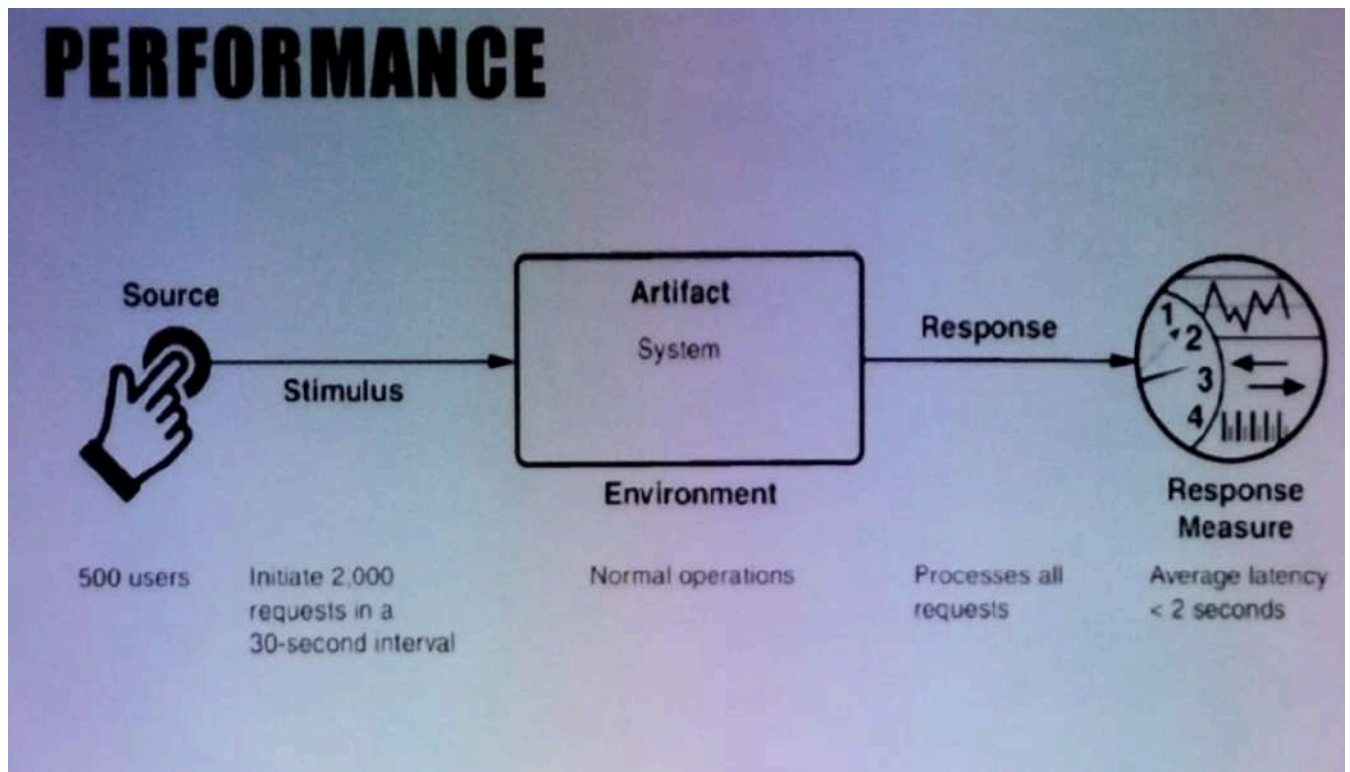
Explica que desde siempre el rendimiento fue un factor fundamental en el desarrollo y que no hay otro atributo de calidad que sea tan importante como este.

Está ligado directamente a la escalabilidad, aumentar la capacidad de trabajo en teoría debería mejorar el rendimiento.

Es mejor enfocarse en problemas que consuman mucho tiempo de cálculo en el sistema antes que en mejoras pequeñas. Hoy en día con la cantidad de tecnología que existe, es mejor optimizar el tiempo de cálculo.

Cuando se producen eventos —interrupciones, mensajes, solicitudes de usuarios u otros sistemas, o eventos de reloj que marcan el paso del tiempo— el sistema, o algún elemento del mismo, debe responder a ellos a tiempo. Caracterizar los eventos que pueden ocurrir (y cuándo pueden ocurrir) y la respuesta temporal del sistema o elemento a dichos eventos es la esencia del análisis del rendimiento.

EJEMPLO



- el estímulo es que se inician 2000 peticiones en un rango de 30 segundos
- en ese ambiente debería poder procesar todas las peticiones sin problemas

TÁCTICAS

- **Objetivo:** asegurar que el sistema genere una respuesta dentro de restricciones de **tiempo** o **recursos**, reduciendo la **latencia** y el **consumo de recursos**.
- **Factores incidentes en la respuesta**
 - **Tiempo de procesamiento**
 - Ocurre cuando el sistema está trabajando activamente en una respuesta. Implicados: CPU, memoria, almacenamiento, red, hilos, buffers.
 - Cada paso de procesamiento suma latencia. A mayor uso → riesgo de saturación.
 - **Tiempo bloqueado (bloqueos por recursos)**
 - **Contención de recursos:** múltiples clientes compiten por un mismo recurso.
 - **Indisponibilidad de recursos:** recurso fuera de línea o fallido.
 - **Dependencia de otros cálculos:** espera a resultados de otro componente (local o remoto)
- **Categorías de tácticas**
 - **Controlar la demanda de recursos:** reducir la cantidad de trabajo o uso de recursos requerido.
 - **Gestionar los recursos disponibles:** usarlos de manera más eficiente para manejar la demanda.

3)

-

-tratar de agregar más nodos, procesamiento concurrente

Del libro:

Tiempo de procesamiento y consumo de recursos: El sistema consume recursos, en este caso el recurso es tiempo. En el caso del hardware los recursos son CPU, memoria, ancho de banda, etc.

Tiempo de bloqueo y contención de recursos: Se puede producir Deadlock (sistemas operativos). Deadlock, explica como los clientes pueden requerir el mismo recurso y esperan uno que requiere otro hasta obtener ese.

Disponibilidad de recursos. Incluso sin contención, el cálculo no puede continuar si un recurso no está disponible. La falta de disponibilidad puede deberse a que el recurso esté ocupado o a una falla del componente por cualquier motivo.

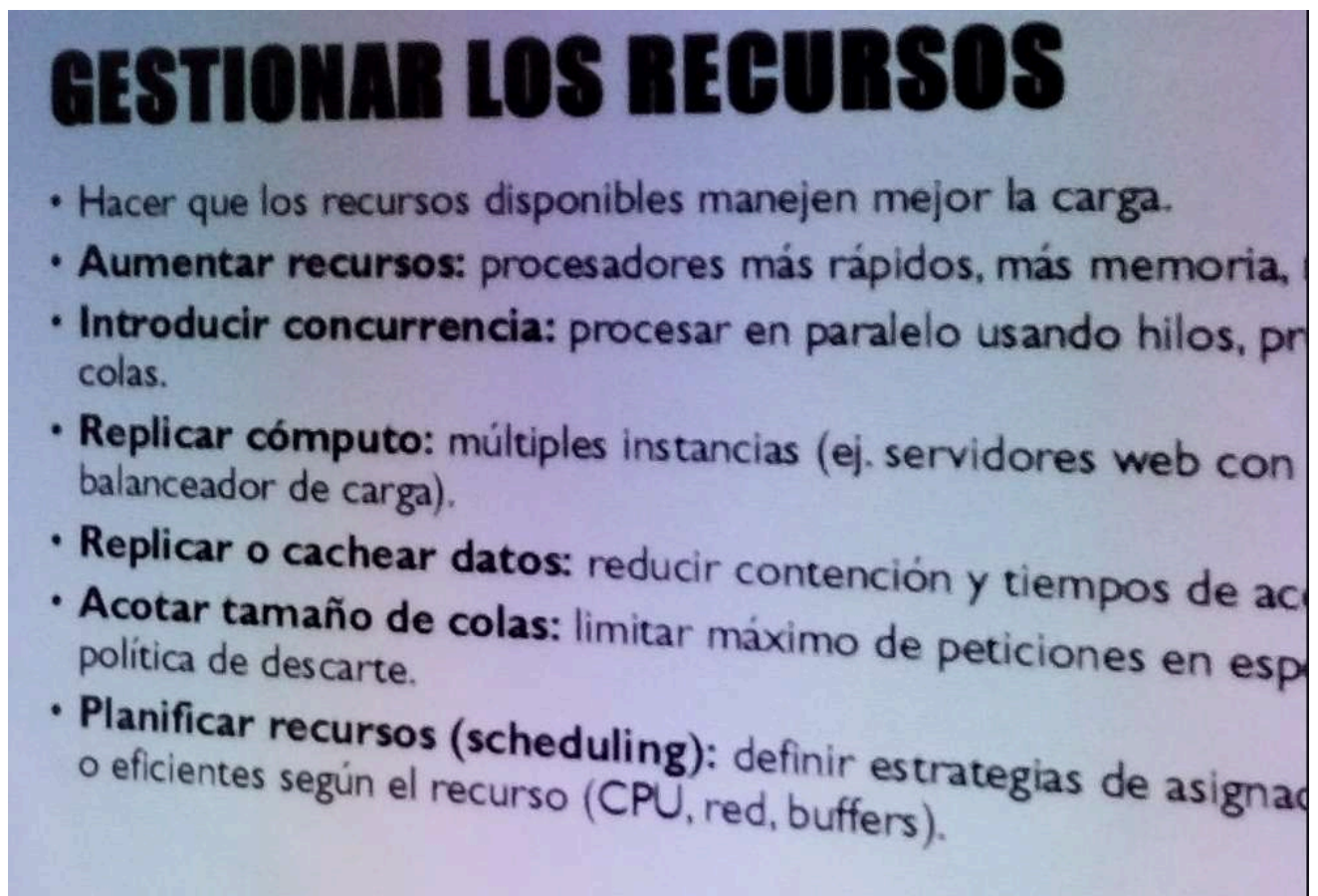
Dependencia de otros cálculos. Un cálculo puede tener que esperar porque debe sincronizarse con los resultados de otro cálculo o porque está esperando los resultados de un cálculo que inició. Si un componente llama a otro componente y debe esperar a que este responda, el tiempo puede ser significativo si el componente llamado se encuentra en el otro extremo de una red (en lugar de estar ubicado en el mismo procesador), o si el componente llamado está sobrecargado.

Hay 2 categorías que diferentes tácticas: Control Resource Demand y Manage Manage Resources

Controlar la demanda de recursos (Control Resource Demand)

- Manejando los mensajes: Una manera de reducir el trabajo es reduciendo la cantidad de mensajes que el sistema puede recibir en un periodo de tiempo.
- Limitar la respuesta de eventos: En vez de intentar responder los eventos ni bien llegan, los pone en cola. Esta cola tiene un tamaño determinado y cuando se llena deja de recibir mensajes o los descarta.
- Priorizar eventos: Si hay suficientes recursos para procesar todo mejor, sino ignora los de menor prioridad. Puede establecer un esquema de prioridades que los clasifique según la importancia de atenderlos.
- Reducir sobrecarga computacional:
 - Reducir la redirección: Reduce la cantidad de intermediarios (cadena de componentes) para obtener un resultado con menor latencia.
 - Tratar de poner recursos que se comunican frecuentemente en el mismo lugar para evitar latencia en la red.
 - Limpiezas periódicas: Se eliminan recursos que quedaron ineficientes. Básicamente reiniciar y que se borre la caché o lo que está virtualizado.
- Limitar el tiempo de respuesta. El costo suele ser una menor precisión en el cálculo.
- Incrementar la eficiencia del uso de recursos: Esto sería mejorar los algoritmos para que se reduzca la latencia y el tiempo de cálculo.

Gestionar los Recursos (Manage Resources)



Del libro:

- Aumentar los recursos o mejorarlos. (El costo suele ser un factor a considerar al elegir los recursos, pero aumentarlos es, en muchos casos, la forma más económica de obtener una mejora inmediata.)
- Introducir concurrencia o paralelismo.
- Tener varias instancias que calculan lo mismo: En vez de tener 1 solo elemento que calcula, tener varios si ese cálculo es muy requerido.
- Mantener copias de los datos: Implementar caché o alguna memoria que evite tener que volver a buscar datos necesarios para hacer cálculos.
- Implementar colas de menor tamaño. (Esta táctica suele combinarse con la de limitar la respuesta a eventos.)
- Agendar o reservar recursos (puede producir deadlock). Como arquitecto, su objetivo es comprender las características de uso de cada recurso y elegir la estrategia de programación que sea compatible con él.

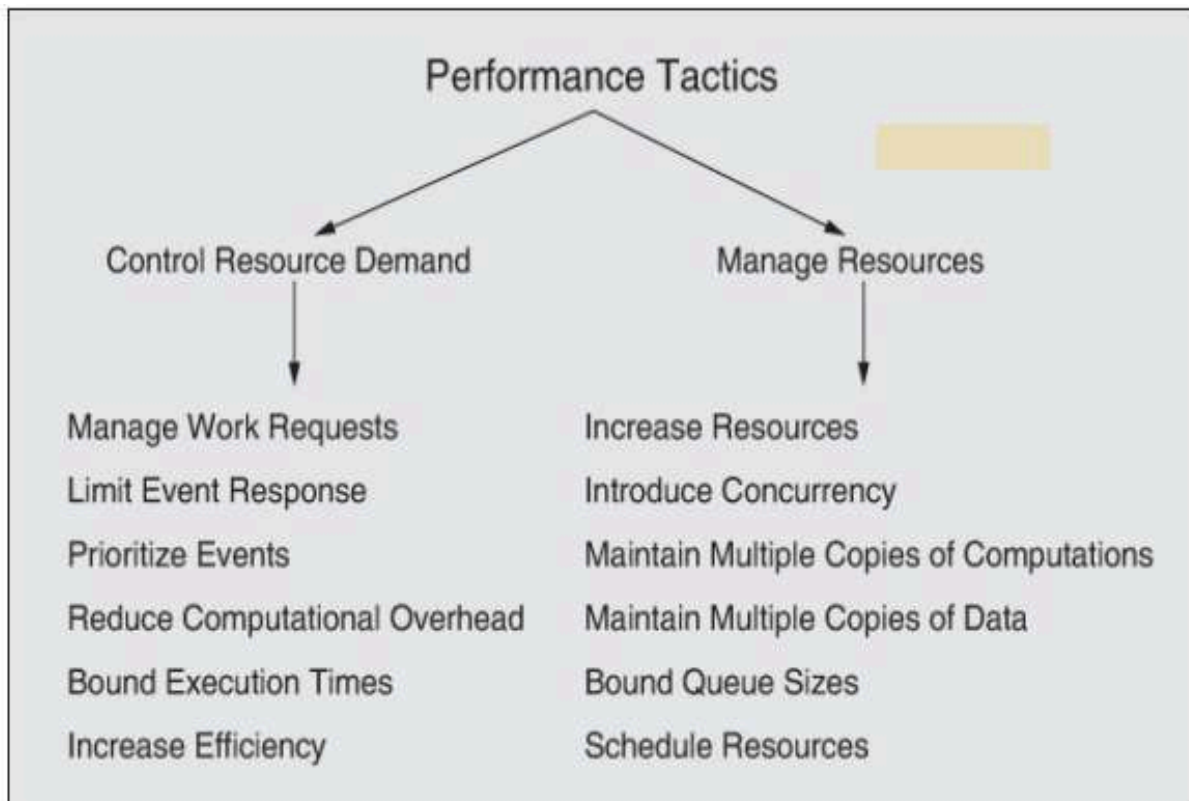


Figure 9.3 *Performance tactics*

PATRONES DE GESTIÓN DE RECURSOS

GESTIONAR LOS RECURSOS

- **Service Mesh**
 - optimización de comunicación en microservicios.
- **Load Balancer**
 - distribución eficiente de carga.
- **Throttling**
 - control de demanda.
- **Map-Reduce**
 - explotación del paralelismo en grandes datos.

4) lo vamos a escuchar en el contexto de big data

-los patrones de performance son más difíciles de entender que los patrones de modificabilidad

Del libro:

Service Mesh (Malla de Servicios)

El patrón de malla de servicios se utiliza en arquitecturas de microservicios.

La característica principal de la malla es un **sidecar**, un tipo de proxy que acompaña a cada microservicio y que proporciona capacidades de gran utilidad para abordar aspectos independientes de la aplicación. Un sidecar se ejecuta junto a cada microservicio y gestiona toda la comunicación y coordinación entre servicios.

Este enfoque permite a los desarrolladores separar la funcionalidad (la lógica de negocio principal) del microservicio de la implementación, la gestión y el mantenimiento de aspectos transversales, como la autenticación y autorización, el descubrimiento de servicios, el balanceo de carga, el cifrado y la observabilidad.

Beneficios:

- Permite a los desarrolladores de la lógica de negocio centrarse únicamente en ese aspecto.
- Reduce el tiempo de comunicación entre el servicio y sus utilidades, ya que la comunicación no requiere el uso de mensajes de red.
- La malla de servicios puede configurarse para que la comunicación dependa del contexto, simplificando así funciones como las pruebas canary y A/B

Desventajas:

- Aumentan la sobrecarga del sistema.
- Un sidecar suele incluir varias funciones, y no todas serán necesarias en cada servicio o en cada invocación de un servicio.

Load Balancer

Un balanceador de carga es un intermediario que gestiona los mensajes procedentes de un conjunto de clientes y determina qué instancia de un servicio debe responder a dichos mensajes.

La clave de este patrón reside en que el balanceador de carga actúa como un **único punto** de contacto para los mensajes entrantes —por ejemplo, una única dirección IP—, pero luego distribuye las solicitudes a un grupo de proveedores (servidores o servicios) que pueden responder a la solicitud. De esta forma, la carga se puede **equilibrar** entre el grupo de proveedores.

Beneficios:

- Cualquier fallo de un servidor es invisible para los clientes (siempre que queden recursos de procesamiento disponibles).
- La latencia se mantiene baja y más predecible para los clientes.
- Es relativamente sencillo añadir más recursos (más servidores, servidores más rápidos) al conjunto disponible para el balanceador de carga, y ningún cliente tiene por qué ser consciente de ello.

Desventajas:

- El algoritmo de balanceo de carga debe ser muy rápido; de lo contrario, podría contribuir a problemas de rendimiento.

- El balanceador de carga es un posible cuello de botella o punto único de fallo, por lo que a menudo se replica (e incluso se balancea la carga).

Throttling

Se utiliza para limitar el acceso a algún recurso o servicio importante. En este patrón, normalmente hay un intermediario —un limitador— que supervisa las solicitudes al servicio y determina si se puede atender una solicitud entrante.

Beneficios:

- Al limitar las solicitudes entrantes, se pueden gestionar de forma eficiente las variaciones en la demanda. De esta manera, los servicios **nunca se sobrecargan**; se pueden mantener en un punto óptimo de rendimiento donde gestionan las solicitudes de forma eficiente.

Desventajas:

- La lógica de limitación de solicitudes debe ser muy rápida; de lo contrario, puede contribuir a problemas de rendimiento.
- Si la demanda de los clientes supera regularmente la capacidad, los buffers deberán ser muy grandes, o existe el riesgo de perder solicitudes.
- Este patrón puede ser difícil de añadir a un sistema existente donde los clientes y los servidores están estrechamente acoplados.

MapReduce

El patrón MapReduce realiza de forma eficiente una ordenación distribuida y paralela de un conjunto de datos grande y proporciona al programador un medio sencillo para especificar el análisis que se va a realizar.

Específicamente diseñado para brindar un alto rendimiento a un tipo específico de problema recurrente: ordenar y analizar un conjunto de datos de gran tamaño. Este problema lo experimenta cualquier organización que maneje grandes volúmenes de datos.

Tiene 3 partes:

1. Fase MAP (Mapeo): Filtrar y Clasificar
2. Fase SHUFFLE (Mezcla y Ordenación)
3. Fase REDUCE (Reducción): Analizar y Resumir

Beneficios:

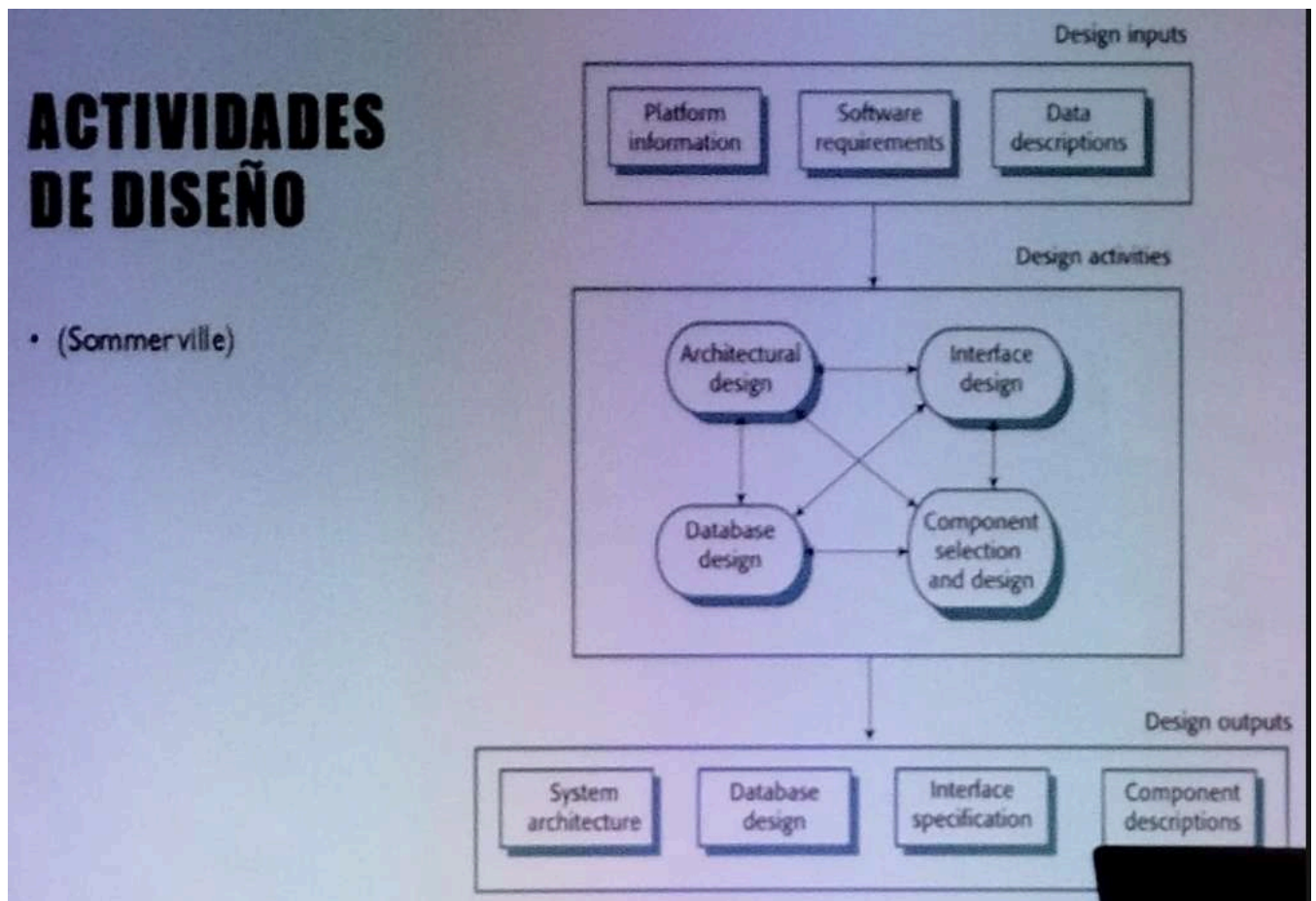
- Los conjuntos de datos extremadamente **grandes** y sin ordenar se pueden analizar eficientemente mediante el aprovechamiento del **paralelismo**.
- El **fallo** de cualquier instancia tiene un **impacto mínimo** en el procesamiento, ya que map-reduce suele dividir los grandes conjuntos de datos de entrada en muchos más pequeños para su procesamiento, asignando cada uno a su propia instancia.

Desventajas:

- Si no se dispone de grandes conjuntos de datos, la **sobrecarga** que supone el patrón map-reduce no se justifica.
- Si no se puede **dividir** el conjunto de datos en subconjuntos de tamaño similar, se pierden las ventajas del paralelismo.
- Las operaciones que requieren múltiples reducciones son **complejas** de orquestar.

CLASE - 10/09

ACTIVIDADES DE DISEÑO



- los primeros dos items son los que más impactan en el diseño de la arquitectura
- el diseño de la arquitectura se conecta principalmente con las interfaces(el libro habla sobre todo de interfaces con componentes)
- si existen componentes ya realizados seleccionar esos que nos sirven a nosotros, sino desarrollarlos

DISEÑO ARQUITECTÓNICO

DISEÑO ARQUITECTÓNICO

- Se ocupa de comprender cómo debe organizarse un sistema de software y diseñar la estructura general de ese sistema.
- Nexo crítico entre el diseño y la Ingeniería de Requisitos, ya que identifica los principales elementos estructurales de un sistema y las relaciones entre ellos.

2) el diseño de la arquitectura es el paso siguiente a la Ingeniería de Requisitos, es la puerta de entrada al diseño. Pensar los grandes elementos que vamos a tener en nuestro sistema

(otro power)

DISEÑO ARQUITECTÓNICO

- La arquitectura **en lo pequeño** se ocupa de la arquitectura de los programas individuales.
- La arquitectura **en grande** se ocupa de la arquitectura de sistemas empresariales complejos que incluyen otros sistemas, programas y componentes de programas.

1) cuando nos enfocamos en la organización que van a tener los programas o el sw

2) cuando estoy pensando en arquitectura de sistemas distribuidos

Del libro:

Se pueden realizar 2 descomposiciones de arquitectura, la cual una puede ser utilizada para mostrar a los stakeholders y la otra para hacer énfasis en componentes y sus elementos internos.

1. La arquitectura a **pequeña escala** se ocupa de la arquitectura de programas individuales. En este nivel, nos interesa la forma en que un programa individual se descompone en componentes. Este capítulo(6) se centra principalmente en las arquitecturas de programas.

2. La arquitectura a **gran escala** se ocupa de la arquitectura de sistemas empresariales complejos que incluyen otros sistemas, programas y componentes de programas.

Estos sistemas empresariales pueden estar distribuidos en diferentes computadoras, que pueden ser propiedad de diferentes empresas y estar gestionadas por ellas. (La arquitectura a gran escala se trata en los capítulos 17 y 18).

Bass sugiere que diseñar y documentar explícitamente el software tiene 3 ventajas:

1. Comunicación con los stakeholders.
2. Análisis del sistema
3. Reutilización a gran escala

Las aparentes contradicciones entre la teoría arquitectónica y la práctica industrial surgen porque existen dos maneras en que se utiliza un modelo arquitectónico de un programa:

1. Como medio para fomentar el **debate** sobre el diseño del sistema: Es un modelo abstracto y de alto nivel. Sirve para comunicarse con los stakeholders (interesados) y para la planificación de proyectos. Al no tener detalles, permite que todos entiendan el sistema en su conjunto y facilita la asignación de tareas de desarrollo.
2. Como medio para **documentar** una arquitectura diseñada: El objetivo aquí es generar un modelo completo del sistema que muestre los diferentes componentes del mismo, sus interfaces y sus conexiones. El argumento a favor de dicho modelo es que una descripción arquitectónica tan detallada facilita la comprensión y la evolución del sistema.

DECISIONES DEL DISEÑO ARQ.

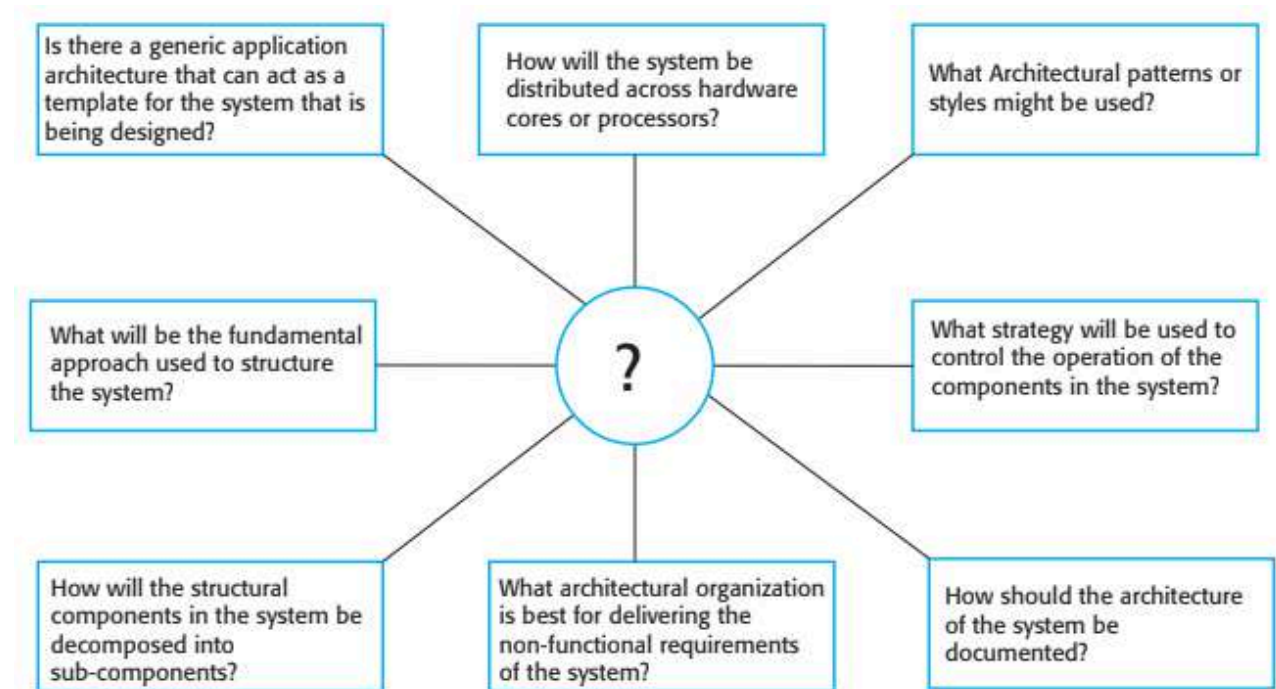


Figure 6.2 Architectural design decisions

(no es necesario que nos enredemos tantos con estas preguntas)

-como va a estar el sistema distribuido en los procesadores o hw? (esto puede pertenecer a la vista de Asignación)

- qué estrategia será utilizada para controlar la operación del sistema?
- como se debería documentar la arquitectura del sistema?
- cuál es la mejor organización para los requisitos no funcionales que voy a tener?
- qué patrones arquitectónicos o estilos podemos utilizar? a través de los atributos no funcionales

Del libro:

En resumen, no se suele utilizar un modelado tan detallado para la arquitectura porque es un trabajo muy complicado y caro.

¿Existe una arquitectura genérica que pueda actuar como plantilla?

¿Estamos construyendo un sistema común (como un sitio web de comercio electrónico) para el cual ya existe una arquitectura de referencia probada? No hay necesidad de reinventar la rueda.

¿Cómo se distribuirá el sistema entre los núcleos o procesadores del hardware?

¿Será una sola aplicación monolítica? ¿O un sistema distribuido (como microservicios) que se ejecuta en múltiples servidores? ¿Cómo aprovechará el paralelismo?

¿Qué patrones o estilos arquitectónicos se utilizarán?

Esta es una decisión clave. ¿Usaremos un patrón de Capas, Cliente-Servidor, Publicador-Suscriptor, Microservicios, etc.? (Muchos de los que has estado revisando).

¿Qué estrategia se utilizará para controlar el funcionamiento de los componentes?

¿Cómo fluye el control? ¿Habrá un "controlador" central que gestione el flujo, o los componentes reaccionarán a eventos de forma independiente (como en el patrón Publicador-Suscriptor)?

¿Cómo se debe documentar la arquitectura del sistema?

¿Cómo comunicaremos el diseño al equipo de desarrollo y a futuros arquitectos? ¿Usaremos vistas de "4+1", diagramas UML, o documentos de decisión (ADRs)?

¿Qué organización arquitectónica es la mejor para cumplir con los requisitos no funcionales?

Esta es una de las preguntas más importantes. Si el requisito no funcional clave es el rendimiento, podríamos elegir un Patrón de Balanceo de Carga. Si es la modificabilidad, elegiremos tácticas de bajo acoplamiento.

¿Cómo se descompondrán los componentes estructurales en subcomponentes?

Una vez que identificamos un componente grande (ej. "Gestión de Pedidos"), ¿cómo lo dividimos en partes más pequeñas y manejables?

¿Cuál será el enfoque fundamental utilizado para estructurar el sistema?

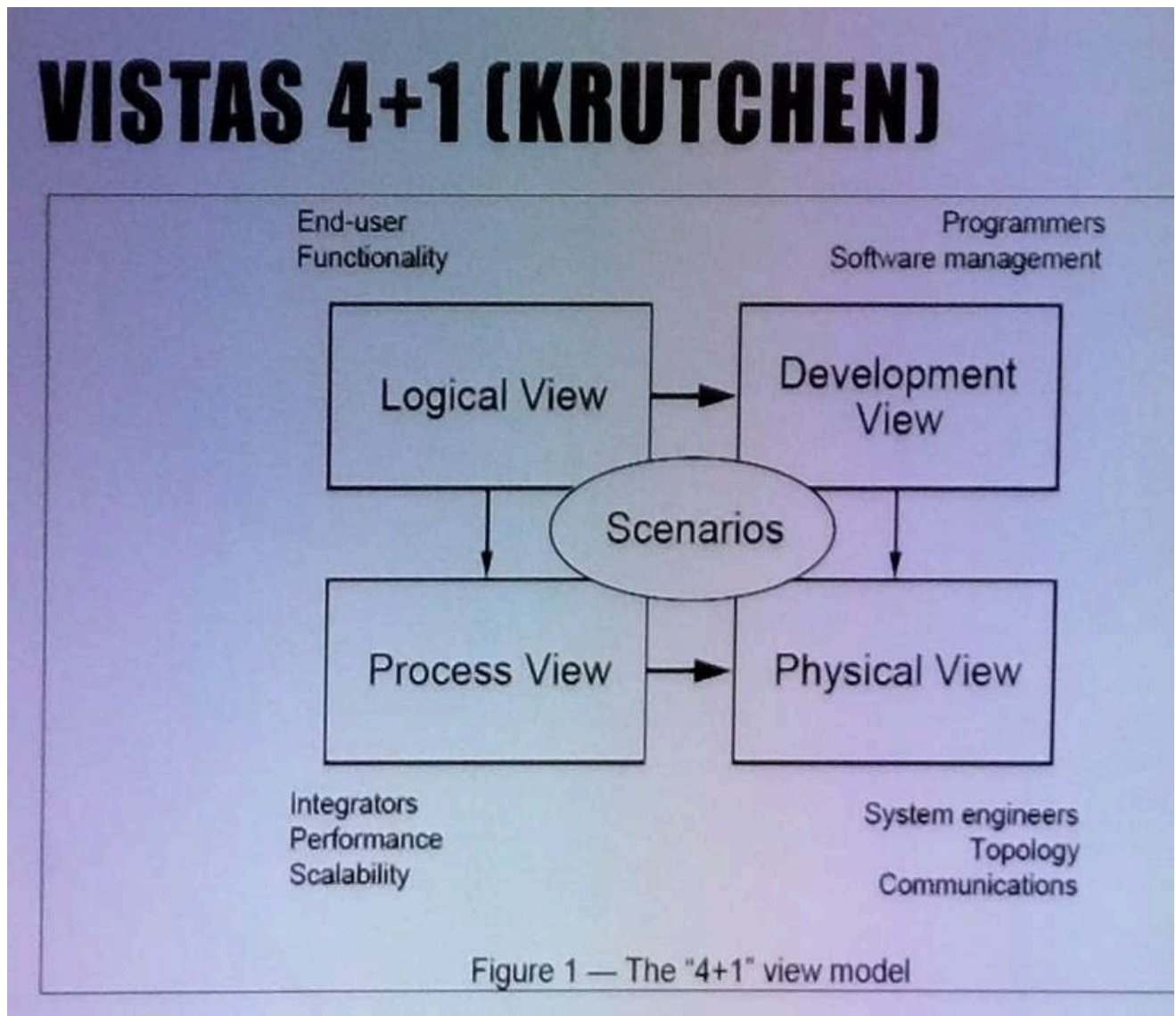
Similar a los patrones, pero más fundamental. ¿Será un sistema orientado a repositorios (con una base de datos central) o un sistema de flujo de datos (pipe-and-filter)?

No hay fórmula mágica para diseñar una arquitectura, pero sí se pueden utilizar (dependiendo el atributo de calidad que quieras priorizar o el requisito no funcional que más impacto tenga en tu sistema) un estilo de arquitectura diferente.

1. Si el **performance** es una prioridad entonces se debería priorizar una arquitectura que minimice la cantidad de operaciones críticas en un pequeño número de componentes y que estos estén en una misma computadora antes que en la red.
2. Si la **security** (frente a ataques o fallos intencionales) es una prioridad, se debería utilizar una arquitectura con una estructura en capas, donde la capa intermedia es la más protegida.
3. Si la **safety** (fallos no intencionales) es una prioridad, entonces se debería utilizar una arquitectura cuyas operaciones estén en un solo componente o en un número reducido de componentes. Esto es porque ante cualquier falla, el sistema puede apagarse y solo quedarían afectados esos componentes.
4. Si la **availability** es una prioridad, entonces es mejor utilizar una arquitectura con varios componentes repetidos para garantizar que si se cae uno o un grupo de ellos exista otro duplicado que cumpla su función.
5. Si la **maintainability** es una prioridad, entonces se debería elegir una arquitectura donde los componentes puedan ser fácilmente reemplazables. Lo ideal sería que los productores de datos estén separados de los que los utilizan.

Obviamente existen conflictos entre arquitecturas, pero eso se soluciona aplicando patrones y tácticas (vistas en Bass) para tratar de cumplir con todos los atributos de calidad necesarios por más que estos hagan conflicto entre sí.

VISTAS 4+1 (KRUTCHEN)



(es muy conocida)

-propone 4 vistas + 1

-Lógica, Despliegue, Proceso, Física

-en la vista de Despliegue como van a distribuir.....

-en la vista Lógica como voy a visualizar mi arquitectura

-en la vista de Procesos tiene que ver cuales son los elementos arquitecturales que se tienen que ver en ejecución

-los Escenarios serían como los Casos de Uso, no va a definir la forma pero indica a qué prestar atención. Voy a tener aquellos cuyo uso me va a determinar los atributos de calidad que al final me van a determinar la arquitectura

-la funcionalidad no moldea la arquitectura, pero no quiere decir que no se tenga en cuenta

-esta organización es la que se utiliza para Proceso Unificado

-está muy relacionado con lo que vimos, en la propuesta de otro autor simplemente

Del libro:

Es imposible representar toda la información relevante acerca del sistema en un solo diagrama, por eso se modelan varias vistas.

Existen cuatro perspectivas arquitectónicas fundamentales, que pueden vincularse mediante casos de uso o escenarios comunes:

Vista lógica: Muestra las abstracciones claves como objetos y clases. Se debería poder relacionar requisitos de sistema con entidades en esta vista.

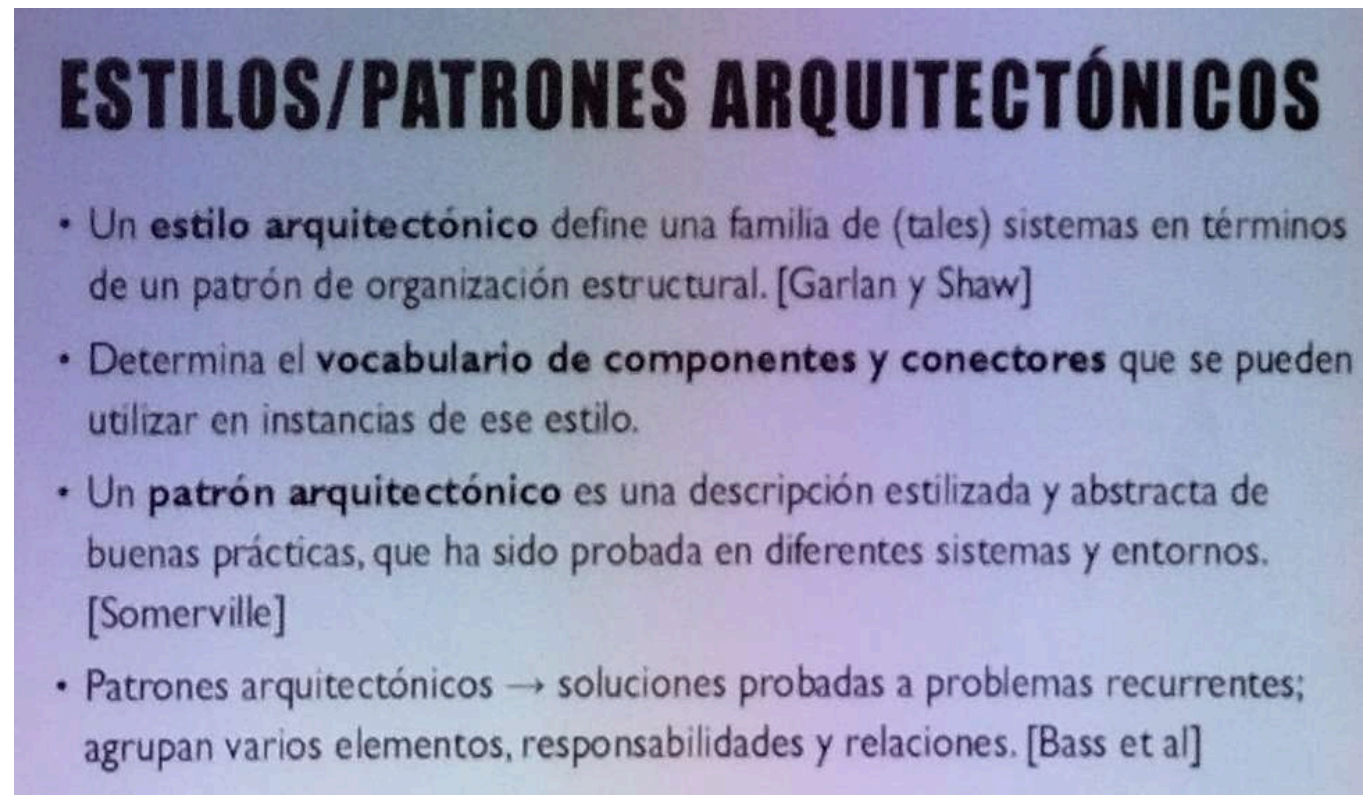
Vista de procesos: Muestra, en tiempo de ejecución, cómo el sistema se compone de procesos. Sirve para comprobar requisitos no funcionales como Rendimiento y Disponibilidad.

Vista de desarrollo: Muestra cómo el software se descompone para el desarrollo, es decir, en componentes implementados por un desarrollador o un grupo de ellos. Es útil para programadores y managers.

Vista física: Muestra el hardware y software y cómo están distribuidos para formar el sistema. Esta vista es útil para los ingenieros en sistemas que planean un despliegue del sistema.

Las vistas se diseñan casi siempre en la Etapa de diseño. Se utilizan para mostrar a los stakeholders o informar de decisiones arquitecturales.

ESTILOS/PATRONES ARQUITECTÓNICOS



ESTILOS/PATRONES ARQUITECTÓNICOS

- Un **estilo arquitectónico** define una familia de (tales) sistemas en términos de un patrón de organización estructural. [Garlan y Shaw]
- Determina el **vocabulario de componentes y conectores** que se pueden utilizar en instancias de ese estilo.
- Un **patrón arquitectónico** es una descripción estilizada y abstracta de buenas prácticas, que ha sido probada en diferentes sistemas y entornos. [Somerville]
- Patrones arquitectónicos → soluciones probadas a problemas recurrentes; agrupan varios elementos, responsabilidades y relaciones. [Bass et al]

(algunos autores intentan definir estos conceptos, pero para esta materia los vamos a tomar como que son iguales. Somerville en últimas versiones a lo que llamaba estilo, cambió todo y lo empezó a llamar patrón)

-un patrón arquitectónico es un problema/solución. Tengo problemas de alto nivel que pueden tener una solución particular

Del libro:

Un patrón arquitectónico es un estilo o abstracción de una buena práctica que fue probada y testeada en diferentes sistemas y entornos. Se debería detallar cuándo es bueno o no utilizar dicho patrón.

MVC (Modelo-Vista-Controlador)

Separa la presentación de la interacción con los datos del sistema. El sistema se estructura en:

Modelo: Maneja los datos del sistema y las operaciones relacionadas con esos datos.

Vista: Define cómo se presenta la información al usuario.

Controlador: Maneja la interacción entre el usuario y la Vista y el Modelo.

Cuándo se usa?

Cuando hay múltiples maneras de ver e interactuar con los datos. También cuando futuros requerimientos para interacción y presentación de datos sean desconocidos.

Ventajas: Permite separar los datos de su presentación. Es decir, permite a los datos ser presentados de distintas maneras.

Desventajas: Puede involucrar código y complejidad adicional en datos e interacciones simples

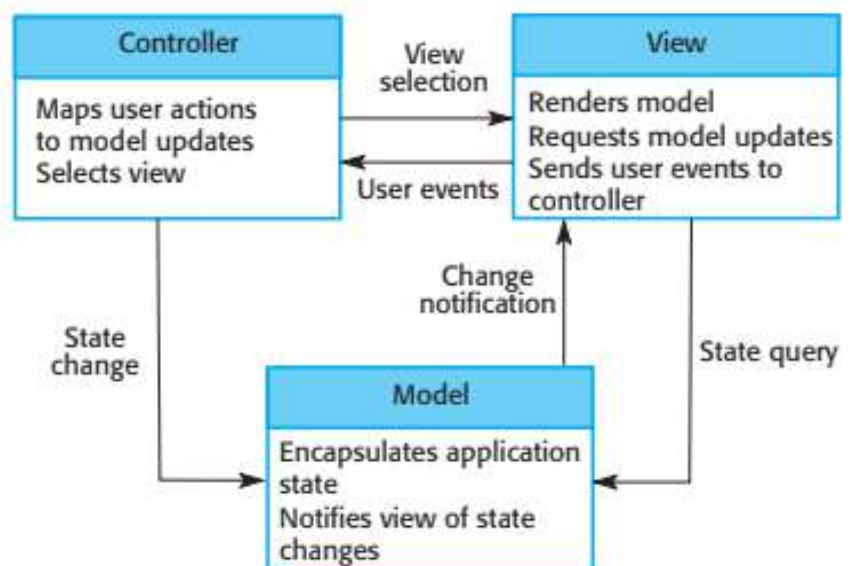
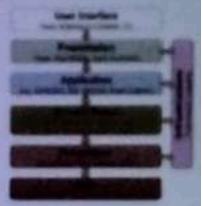


Figure 6.5 The organization of the Model-View-Controller

EN CAPAS (LAYERED)

EN CAPAS (LAYERED)

- Una forma de lograr **separación e independencia**.
- La funcionalidad se organiza en **capas separadas**.
- Cada capa **depende** de la capa inmediatamente inferior.
- Cada capa es **independiente** de las capas superiores.
- Desarrollo **incremental** de sistemas.
- **Cambiable**, una nueva capa puede reemplazar una capa existente
- **Portátil**, ya que los sistemas en capas localizan las dependencias de la máquina, implementaciones multiplataforma.
- Una separación limpia entre capas suele ser difícil.
- El rendimiento puede ser un problema.



- se trata de que en cada capa exista elementos con responsabilidades similares
- normalmente arriba de todo esta la capa de presentación, en el medio va aplicación o lógica de negocio y en el final la capa de datos
- cada capa de arriba depende inmediatamente de la capa inferior, pero no a la inversa. Esto es para que capa se pueda remover y ser reemplazada sin problema, esto permite que tengamos un desarrollo incremental. Debido a que capa podríamos ir resolviendo de a poco
- deben estar bien específicas las interfaces
- no siempre se puede lograr esa separación tan estricta entre capas (a veces suele ser difícil)
- a medida que vamos agregando capas, el rendimiento de como este hecho puede llegar a ser un problema

- este patrón está asociado al atributo de calidad de modificabilidad

Del libro (Somerville):

-Separa el sistema en capas con funcionalidades relacionadas entre cada una. Una capa provee los servicios para la capa superior, entonces las de nivel inferior representan los servicios más importantes del sistema.

-**Se usa cuando** se desean construir nuevas características por encima de sistemas que ya existen. Cuando el desarrollo está dividido en varios equipos y cada uno sea responsable de una capa de funcionalidad. Cuando la seguridad como atributo de calidad es crucial.

-Claramente una **desventaja** es que en la práctica es muy complicado separar claramente las capas y habrá ocasiones en las que capas de nivel superior se comuniquen directamente con las de nivel inferior. Además, el rendimiento puede ser un problema por la latencia entre capas.

-Es ventajoso para sistemas multiplataforma porque solo las capas que estén ligadas al sistema operativo deberán ser reemplazadas

EJEMPLO

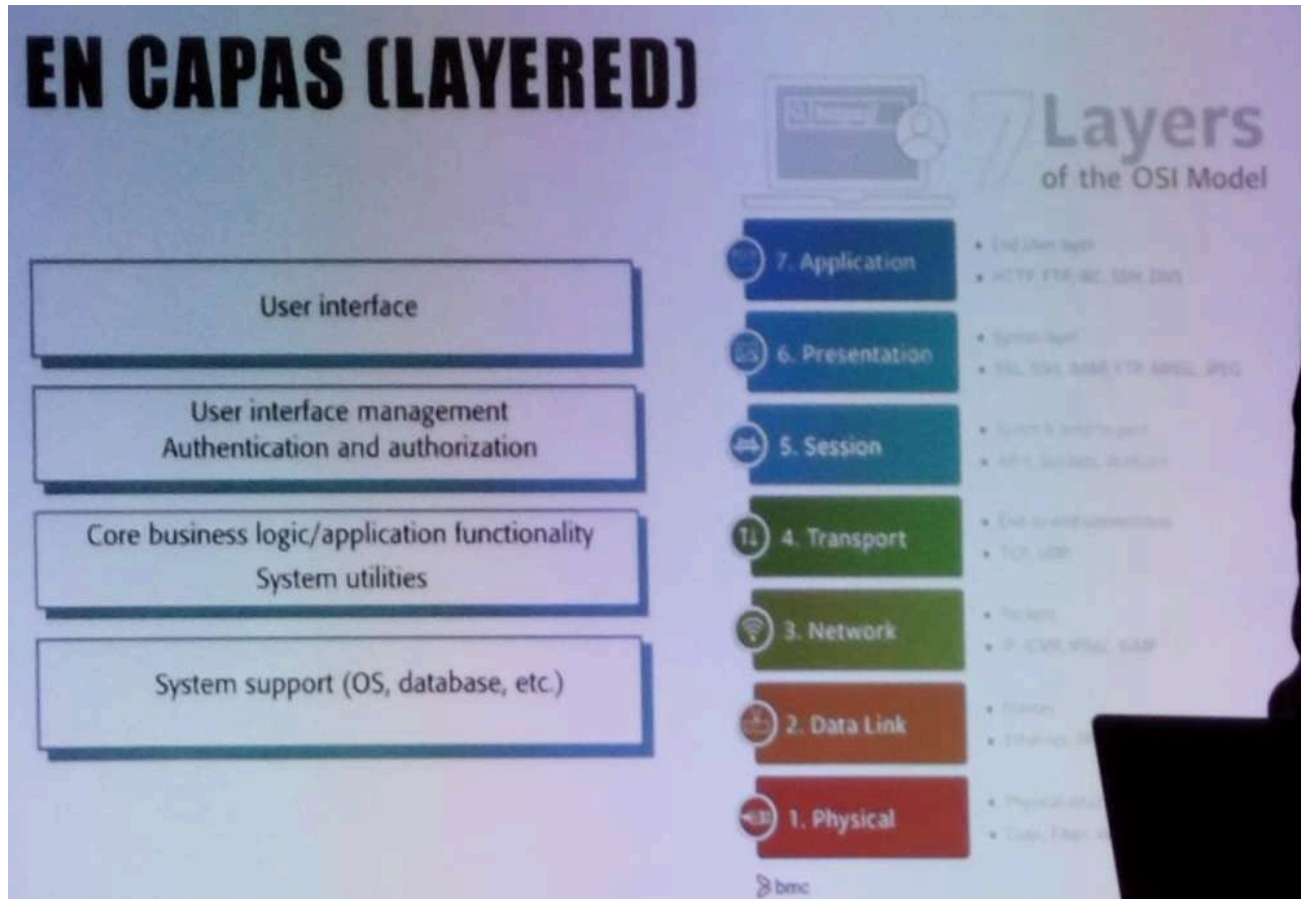
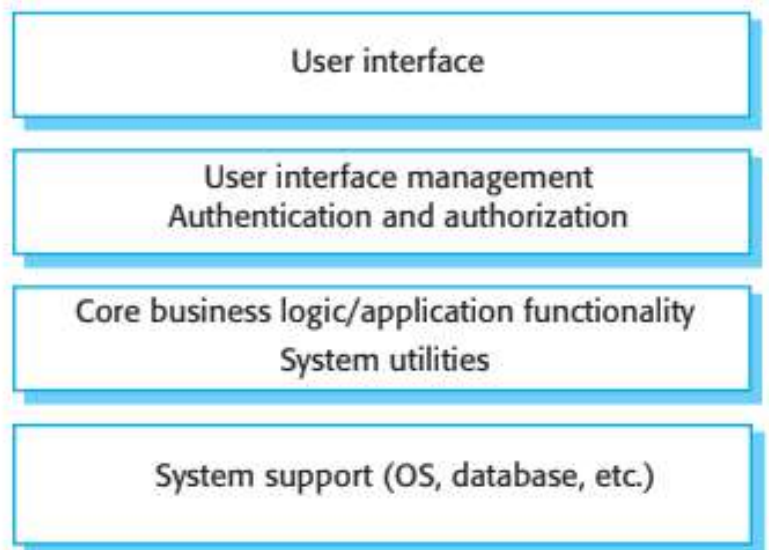
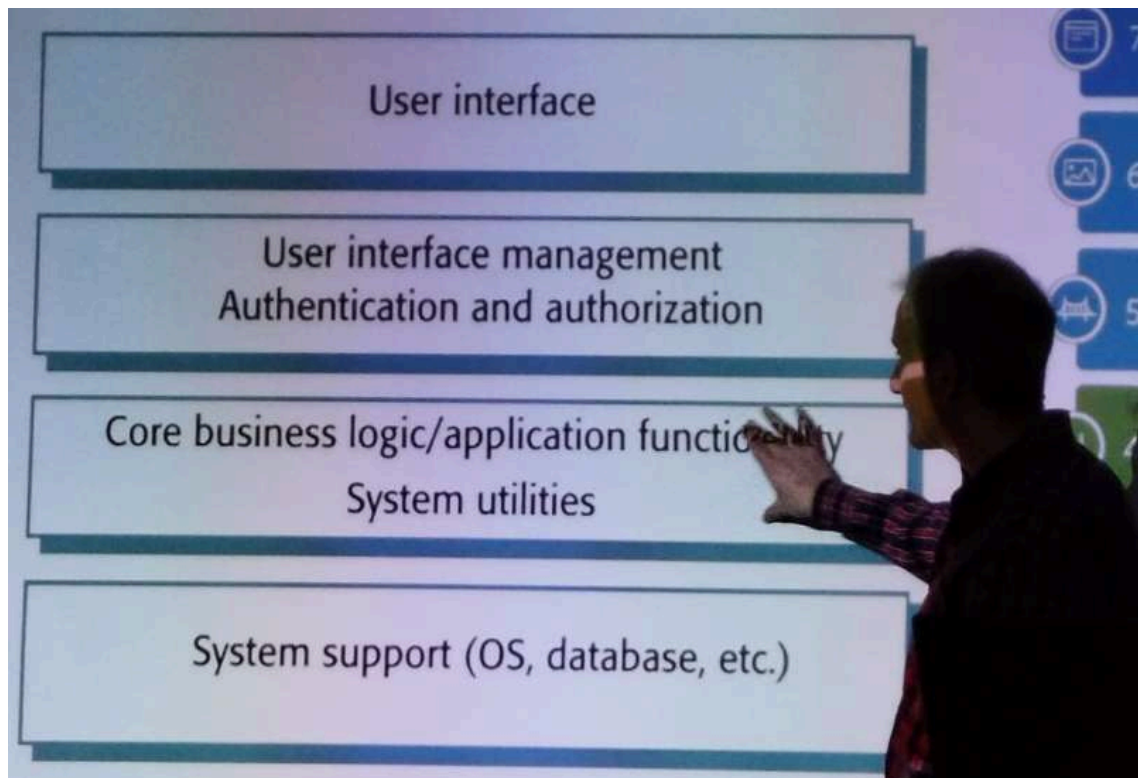


Figure 6.8 A generic layered architecture





-capa de lógica del negocio (esta capa si estuviera bien hecho no debería depender de la base de datos asociada por ejemplo)

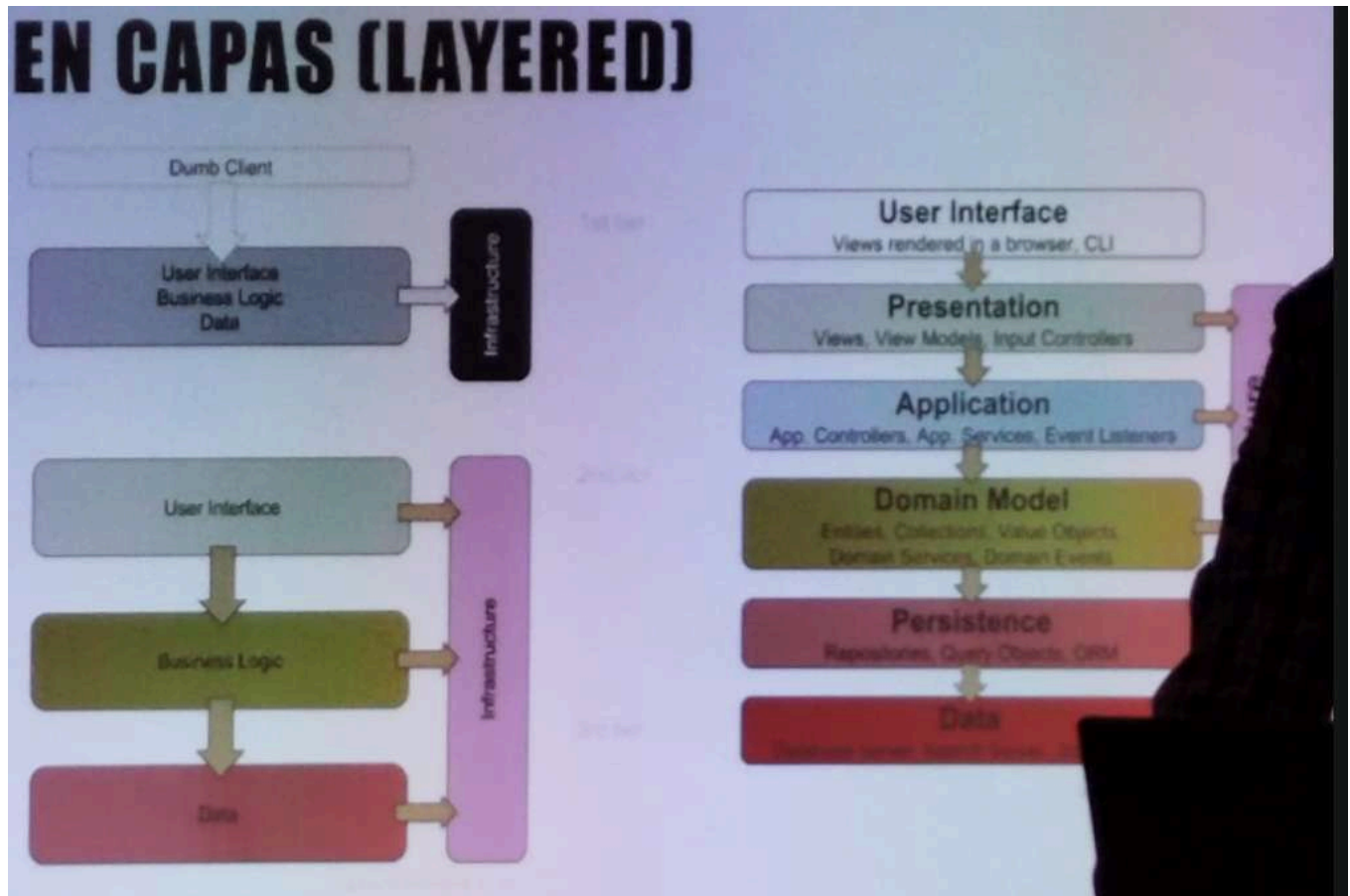
-capa de soporte (más vinculado al sistema operativo, base de datos, etc)

Del libro:

La figura 6.8 muestra un ejemplo de una arquitectura en capas con cuatro capas.

Por supuesto, el número de capas es arbitrario. Cualquiera de las capas de la figura 6.6 podría dividirse en dos o más capas.

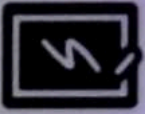
(otro power)



(Ejemplo derecha)

- dentro de la aplicación fijense que tenemos los controladores
- además identifiquen que a su vez está más dividida porque tenemos la interfaz de usuario y la presentación
- Tier normalmente se lo suele asociar como una capa física y layer como lógica.

REPOSITORIO / PIZARRÓN



- Los componentes independientes se comunican a través de un repositorio de datos central (también componente)
- Los componentes independientes no interactúan directamente.
- Conveniente cuando:
 - se generan grandes volúmenes de datos que tengan que ser almacenados durante mucho tiempo (repositorio-pasivo).
 - la inclusión de datos en el repositorio desencadena una acción (pizarrón-activo).
- Todos los datos se pueden administrar de manera consistente.
- El repositorio es un único punto de fallo (afecta a todo el sistema).
- Puede haber ineficiencias por centralizar toda la comunicación.
- Distribuir el repositorio entre varios equipos puede ser difícil.

(no es muy conocido a nivel popular, pero está en la mayoría de bibliografías)

(esto pueden desarrollar en el libro)

-es el que va a encargarse de contener los datos de otros componentes

-la forma de comunicación que existe entre los componentes es a través del Repositorio, no hay comunicación entre los componentes

-cuando ese componente central es del estilo **pasivo**, no emite una señal o no trata de comunicarse se lo conoce como **Repositorio**

-cuando toma un rol **activo** ese componente, ahí se lo conoce como modelo de **Pizarrón**

-todo los datos se pueden guardar de manera consistente porque todos se encuentran en el mismo lugar

-si hay un problema en el repositorio el sistema completo deja de funcionar

Del libro:

Cuando usar?

Se debería usar en sistemas donde haya grandes volúmenes de información y se deban almacenar durante mucho tiempo. También en sistemas guiados por datos donde la inclusión de datos en este repo central activa un trigger o algo.

Ventajas:

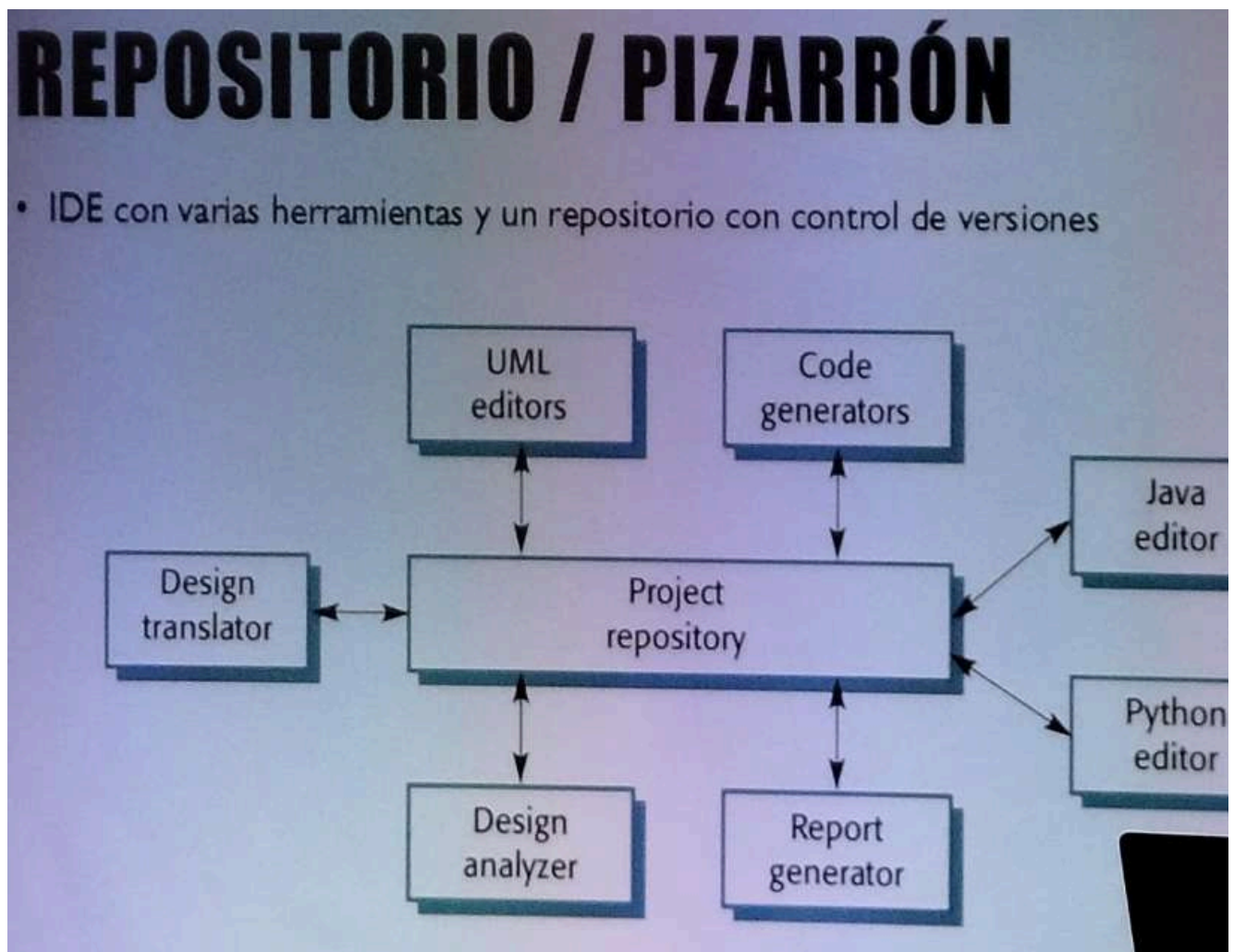
-Los componentes son independientes, no necesitan conocer a los demás y los cambios que realiza en un componente se pueden propagar a todos los demás.

Todos los datos se manejan de forma consistente y es más fácil hacer backup (porque esta todo en un mismo lugar)

Desventajas:

- El repositorio es un punto único de fallo, por lo que los problemas en el repositorio afectan a todo el sistema.
- Puede haber ineficiencias en la organización de toda la comunicación a través del repositorio. (Cuello de botella)
- Distribuir el repositorio en varios ordenadores puede resultar difícil.

EJEMPLO



(para ampliar un poco más de este tema se llama un libro POSA el cual tiene varios volúmenes, dijo que no quiere darlo porque se amplía mucho este tema de arquitectura)

- tengo editores en diferentes lenguajes
- tengo analizadores o reportes basado en los datos que tengo
- es un patrón que da soporte a operaciones no determinísticas

Cliente-servidor

En una arquitectura cliente-servidor, el sistema es presentado como una serie de servicios, donde cada servicio es brindado por un servidor distinto. Los clientes acceden a ese servicio accediendo al servidor.

Cuando usar? Se utiliza cuando los datos de una base de datos compartida deben accederse desde distintas ubicaciones. También se utiliza cuando la carga del sistema varía debido a que los servidores pueden replicarse.

La principal **ventaja** de este modelo es que puede distribuirse por la red. Una funcionalidad puede ser brindada para todos los clientes pero no necesariamente implementada en todos los servicios. Además, el beneficio más importante es la separación e independencia. Se pueden cambiar servidores y servicios sin afectar al resto. O añadir capacidad de procesamiento.

Desventajas:

Cada servidor es susceptible a DDoS o fallos de servidor. El rendimiento es impredecible debido a la latencia de la red y del sistema al mismo tiempo. Se pueden presentar problemas si los servidores son de distintas organizaciones.

Los principales **componentes** de este modelo son:

1. Un conjunto de **servidores** que ofrecen servicios a otros componentes. Ejemplos de servidores que incluyen servidores de impresión que ofrecen servicios de impresión, servidores de archivos que ofrecen servicios de gestión de archivos y un servidor de compilación que ofrece servicios de compilación de lenguajes de programación. Los servidores son componentes de software y varios servidores pueden ejecutarse en el mismo ordenador.
2. Un conjunto de **clientes** que solicitan los servicios ofrecidos por los servidores. Normalmente habrá varias instancias de un programa cliente ejecutándose simultáneamente en diferentes ordenadores.
3. Una **red** que permite a los clientes acceder a estos servicios. Los sistemas cliente-servidor se implementan generalmente como **sistemas distribuidos**, conectados mediante protocolos de Internet.

La arquitectura cliente-servidor está pensada para sistemas distribuidos pero el modelo lógico de servicios independientes que funcionan en distintos servidores pueden implementarse en una misma computadora.

Los clientes deberían conocer los nombres de los servidores disponibles y de los servicios que proveen pero los servidores no necesitan conocer a los clientes que acceden a sus servicios.

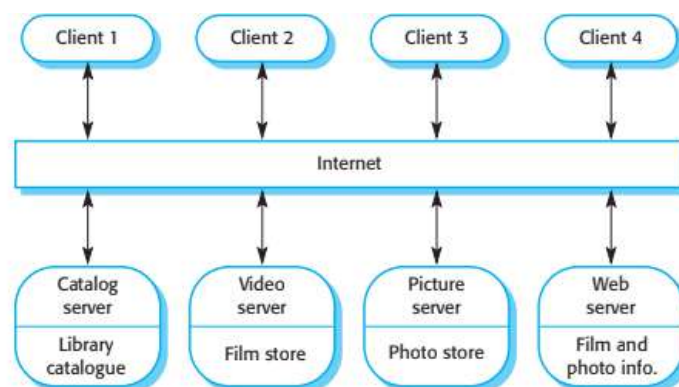


Figure 6.13 A client-server architecture for a film library

PIPES AND FILTERS



Ceci n'est pas une pipe.

- Cada componente tiene un conjunto de entradas y de salidas.
- **Filtro:** produce un resultado aplicando una **transformación local** a los flujos de entrada y calculando de forma incremental (usualmente) para que la salida comience antes de que se consuma la entrada.
- **Tubería:** es un conector entre filtros, conducto para los flujos de datos.
- Los filtros deben ser entidades **independientes**. No comparten estados con otros filtros. No conocen a otros filtros.

-tenemos componentes que nos ayuda hacer transformaciones

-son componentes enfocados en realizar determinada tarea

-la entrada de un componente es la salida de un componente anterior

-no comparten estados con otro filtro, osea que una vez terminó de hacer su tarea pasa a hacer su siguiente tarea. No se conocen entre sí

-se utiliza mucho para la gestión de datos

(otro power)

PIPES AND FILTERS



Ceci n'est pas une pipe.

- Comúnmente utilizado en aplicaciones de procesamiento de datos.
- Fácil de entender y admite la reutilización de la transformación.
- El flujo de trabajo es similar al de procesos de negocio.
- La adición de transformaciones es sencilla. Pueden ser secuenciales o concurrentes.
- El formato de datos debe acordarse entre las transformaciones contiguas.
- Se adapta al procesamiento por lotes y no al interactivo.

-se utiliza mucho para lo que es procesamiento de flujo de trabajos

-no está pensado para ser interactivo con el usuario

-van a ver que algunos patrones que son mas aplicaciones a interactivas o a otro más relacionado a los que son por lotes

Del libro:

El procesamiento de datos en el sistema está organizado de manera que cada componente de procesamiento (filtro) sea discreto y transforme un tipo específico de datos. Los datos son procesados de componente en componente.

Comúnmente utilizado en aplicaciones de procesamiento de datos (por lotes y basadas en transacciones) donde los inputs son procesados en etapas separadas para generar outputs relacionados.

Se utiliza para sistemas donde la interacción con el usuario no es tan importante.

Se originó de las primeras computadoras que solamente servían para procesar datos.

Es común en sistemas de pagos

Ventajas:

-Es fácil de entender y soporta reutilización.

-El flujo de trabajo es parecido al de muchas organizaciones.

-La evolución mediante agregación de transformaciones es fundamental.

-Se puede implementar de manera secuencial o concurrente.

Desventajas:

-El formato de transferencia de datos debe acordarse entre las transformaciones que se comunican.

-Cada transformación debe analizar su entrada y convertir su salida al formato acordado. Esto aumenta la sobrecarga del sistema y puede imposibilitar la reutilización de componentes arquitectónicos que utilizan estructuras de datos incompatibles.

Ejemplo chat gpt

Combinación de los comandos con tuberías

Podemos combinar estos comandos usando tuberías para que la salida de cada comando se convierta en la entrada del siguiente:

```
sh
```

```
cat sample.txt | grep "error" | wc -l
```

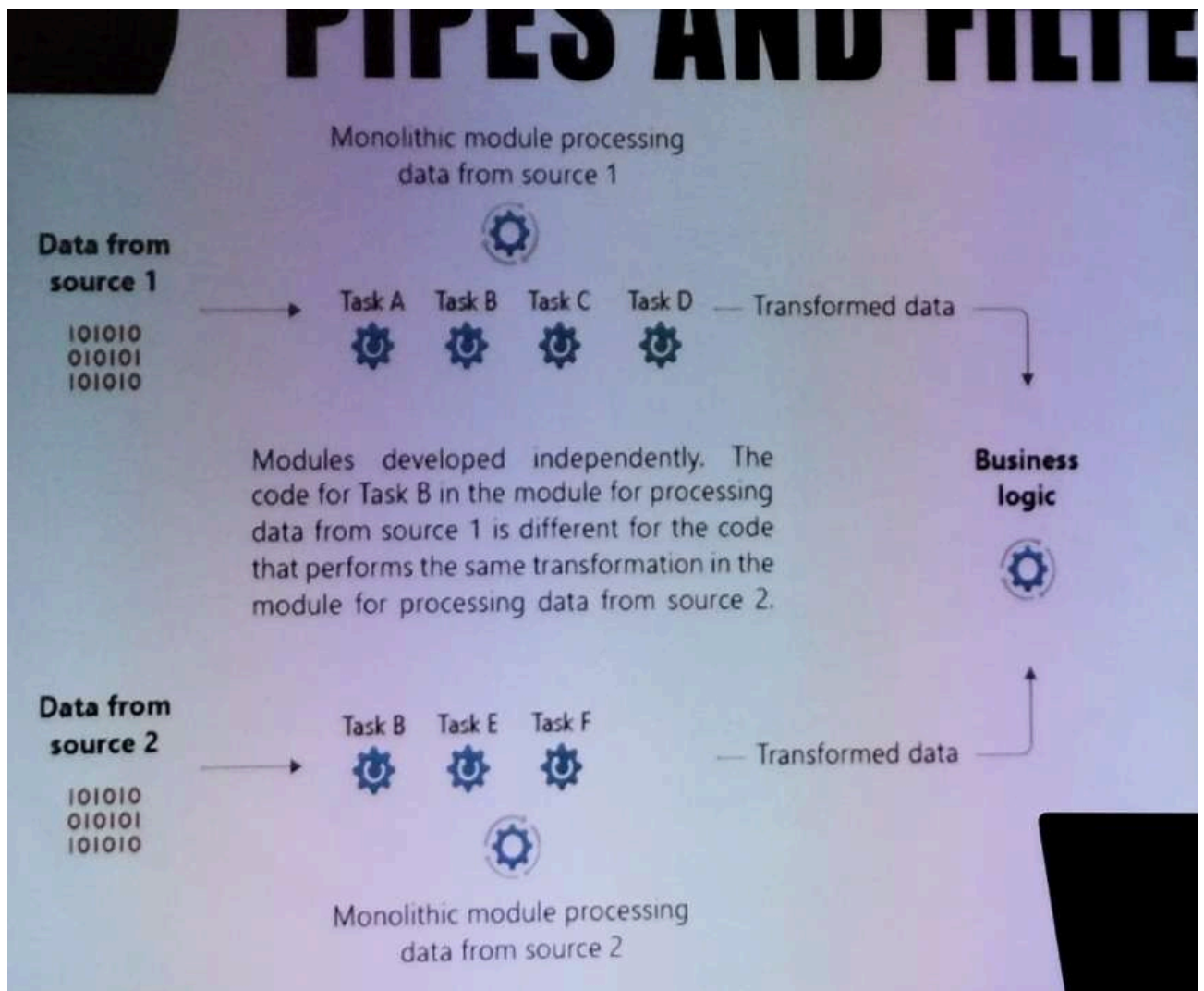
Copiar código

Explicación de los comandos en la tubería

1. `cat sample.txt`: Este comando lee el contenido del archivo `sample.txt` y lo envía a la salida estándar.
2. `grep "error"`: Toma la salida de `cat`, filtra las líneas que contienen la palabra "error" y envía el resultado a la salida estándar.
3. `wc -l`: Toma la salida de `grep` y cuenta el número de líneas, mostrando el resultado.

- las tuberías serían las: |
- este comando lo primero que hace es leer el contenido del archivo
- luego trata de filtrar las líneas que contiene error y filtra el resultado
- hace un conteo de palabras

Ejemplo diagrama



-particularmente microsoft soporta usar este patrón para el filtrado de datos

-donde están las tareas enumeradas esos son un conjunto completo(parece que no salió en la imagen)

-lo que hace ahí justamente es tratamiento de datos, que luego concluye en un destino particular

diagrama2

-ahora lo que hace es tratar de usar los diferentes componentes de esta manera

diagrama3

-la mejor forma podría ser esta, tratando de tener tareas en paralelo

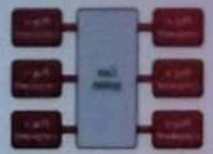
-lo que permite este patrón es ir agregando tuberías

(otro power)

2) cambiar de tamaño las imágenes. Finalmente cuando está lista la imagen la publica en una Red de Distribución de Contenido

-este patrón que se hizo hace 50 años, se sigue utilizando hoy en día plenamente

MICROKERNEL / PLUG-IN



- La comunicación suele ser **punto a punto**, lo que significa que suele ser una invocación de método o una llamada a una función a la clase de punto de entrada del plug-in.
- **Plug-in de tiempo de ejecución** se pueden agregar o quitar en tiempo de ejecución sin tener que volver a implementar el sistema central u otros plug-in.
- **Plug-in basados en compilación** son mucho más sencillos de administrar, pero requieren que se vuelva a implementar toda la aplicación monolítica cuando se modifica, agrega o quita.
- Los plug-in punto a punto se pueden implementar como bibliotecas compartidas (JAR, DLL), nombres de paquetes en Java o espacios de nombres en C#.

-tenemos un sistema central con las funcionalidades básicas que puede tener ese sw

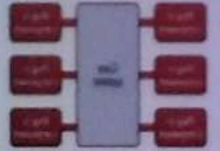
-después puede adaptarse a través de lo que denominamos plug-in

-ejemplo: chrome a través de las extensiones que le vamos agregando

-estos plug-in son llamados a través de funciones o invocaciones

-los basados en tiempos de ejecución permiten mayor modificabilidad debido a que no debo volver a compilar todo el sistema para realizar un cambio.

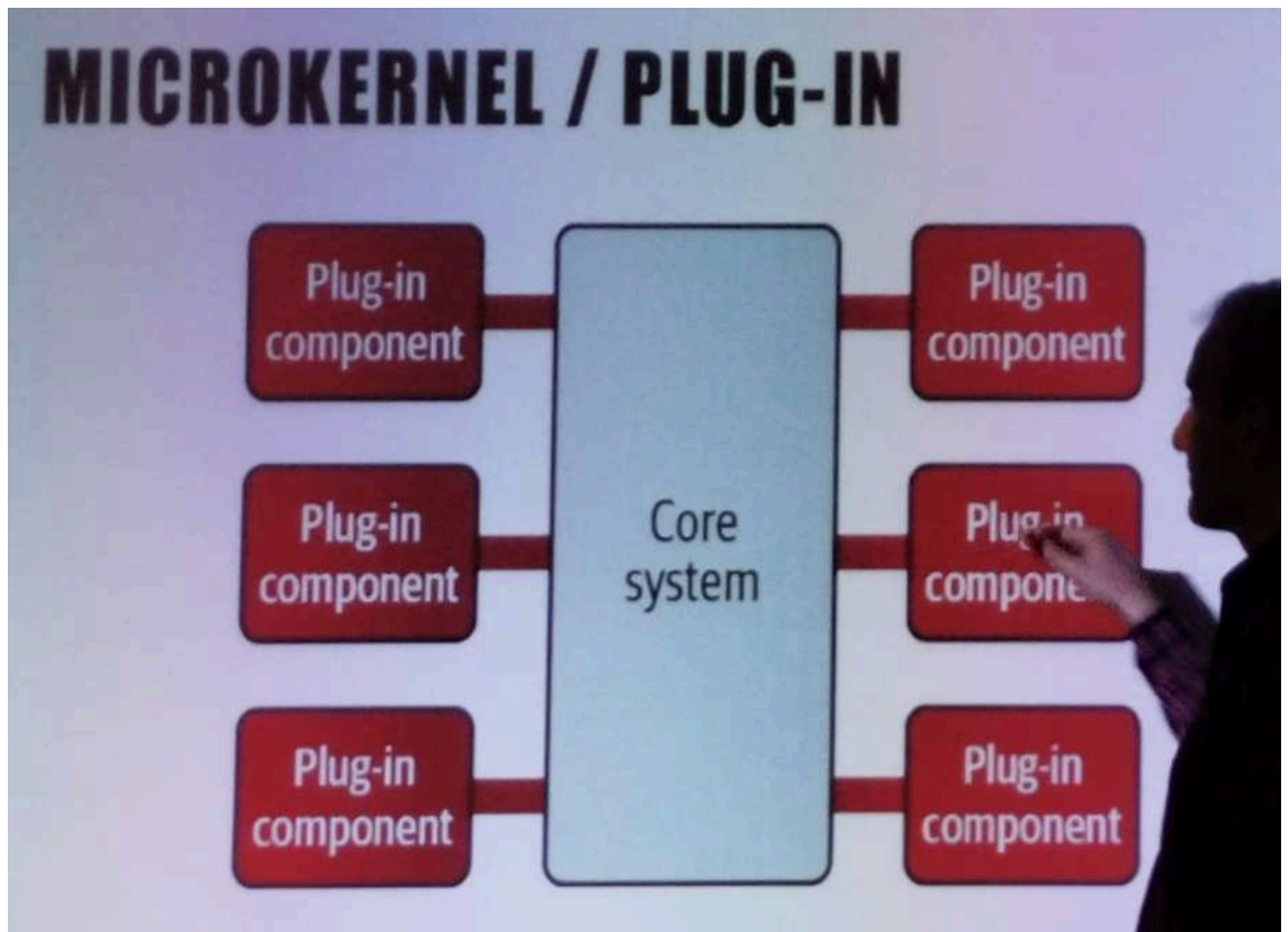
MICROKERNEL / PLUG-IN



- Consta de un **sistema central y plug-ins**, que proporcionan extensibilidad, adaptabilidad y aislamiento de las características de la aplicación y la lógica de procesamiento personalizada.
- **Sistema central:** componente con la funcionalidad mínima requerida para ejecutar el sistema.
- **Plug-ins:** componentes independientes y autónomos que contienen procesamiento especializado, características adicionales y código personalizado destinado a mejorar o ampliar el sistema central.

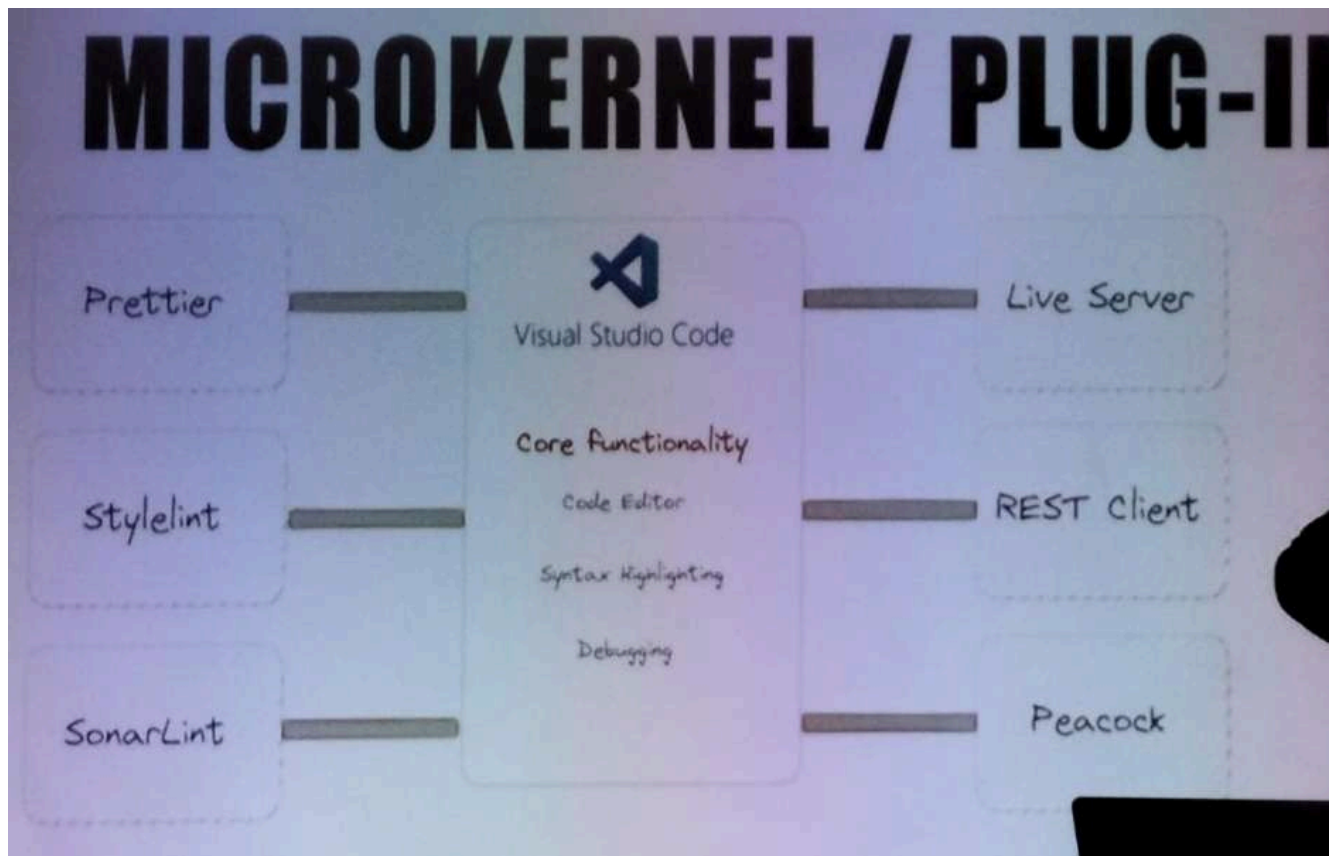
-se parte de un núcleo central donde le voy agregando cosas, en cambio el sistema central ya se parte de un sistema completo en el cual no se piensa en ir agregando cosas
-este sistema central no es solo de datos

EJEMPLO



-Diagrama básico y los componentes que se pueden ir enchufando

EJEMPLO(visual studio)



ejemplo reclamos

-como voy a ir separando por zona lo puedo tener dividido en distintos plug-in